ARTIFICIAL INTELLIGENCE (Q) – MARCH 2026
KECERDASAN TIRUAN (Q) – *FEBRUARI 2026*

INTRODUCTION ON ARTIFICIAL INTELLIGENCE

Pendahuluan Kecerdasan Tiruan

Mohammad Iqbal, S.Si., M.Si., Ph. D.

OUTLINES

- Search Algorithm: BFS, DFS and UCS
- Heuristic Algorithm: Greedy, A*

Question

A **farmer** wants to get his **cabbage**, **goat**, and **wolf** across a river. He has a boat that only holds two. He cannot leave the cabbage and goat alone or the goat and wolf alone. How many river crossings does he need?

- 4

- 5

- 6

- 7

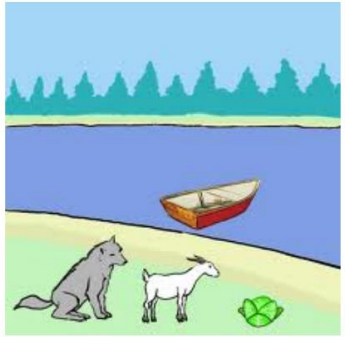
- no solution

https://stanford-cs221.github.io/autumn2023-extra/modules/search/modeling.pdf

Model the Problem

How many different “states”?

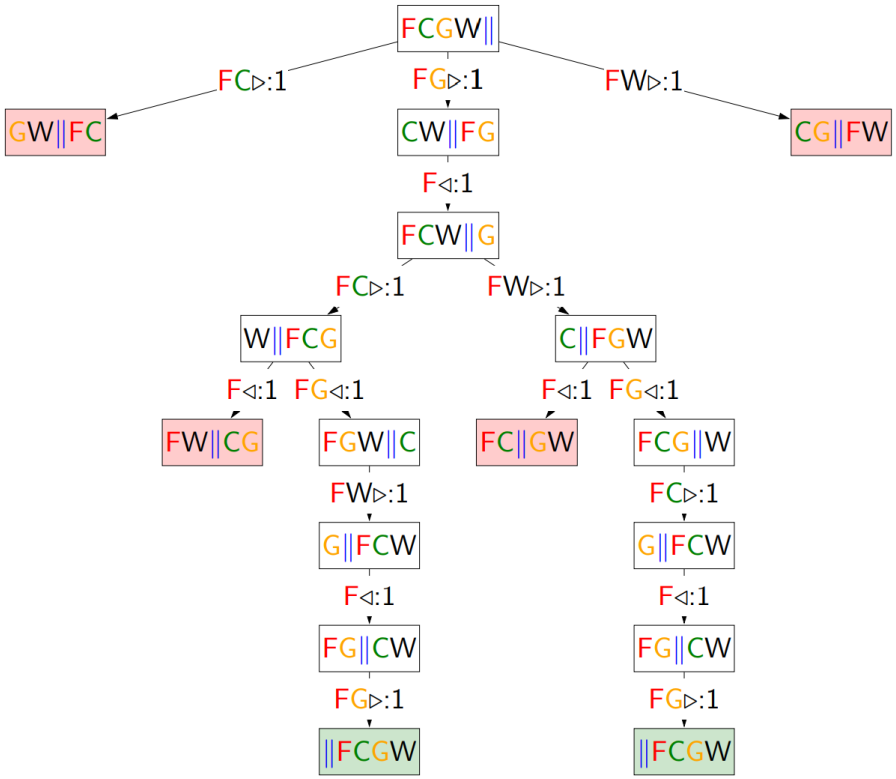
How many different “actions”?



Farmer Cabbage Goat Wolf

F▷	F◁
FC▷	FC◁
FG▷	FG◁
FW▷	FW◁

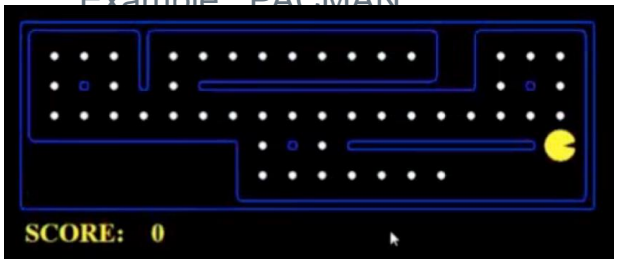
Build a Search Tr



Search Problem

- State space
- Initial state
- Goal test: Given a state, return whether the state is a goal
- Action
- Successor function: Given current state and action, return the new state
- (The cost of an action)

Example: PACMAN



Eat all dots

States: {(x,y), dot booleans}

Actions: NSEW

Successor: update location and possibly a dot boolean

Goal test: dots all false

Go to some destination

States: (x,y) location
Actions: NSEW
Successor: update location only
Goal test: is (x,y)=END

Example: SAINT (Slagle, 1961)

Symbolic Integrator

$$\int \frac{x^4}{(1-x^2)^{5/2}}\,dx \quad = \quad \frac{1}{3}\tan^3(\arcsin x) - \tan(\arcsin x) + \arcsin x$$

States: symbolic expression

Actions: "common techniques"

Successor: the new expression after applying the technique

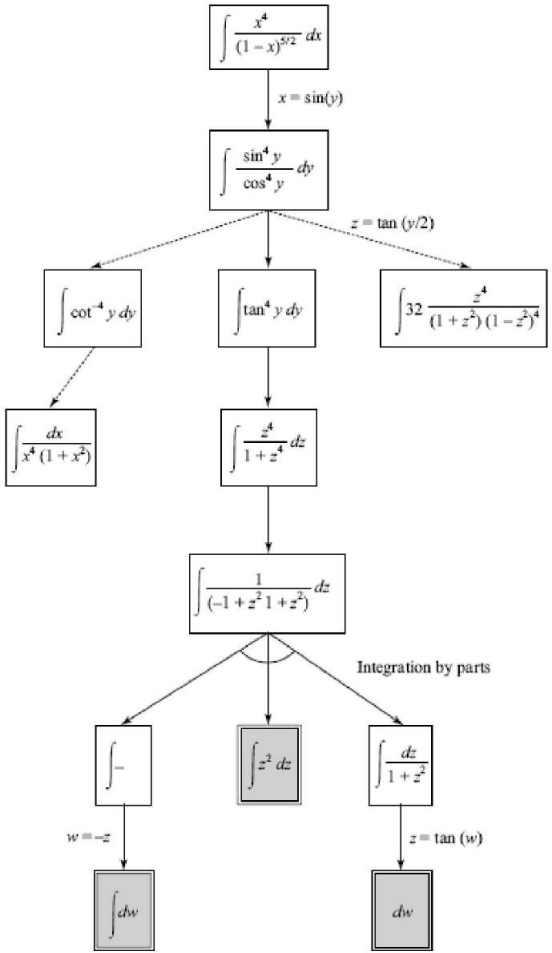
Goal test: whether the expression is in "standard form"

"common technique" examples:

$$\int c\, f(x)\mathrm{d}x = c \int f(x)\mathrm{d}x$$

$$\int f(\tan x)\mathrm{d}x = \int \frac{f(y)}{1+y^2}\,\mathrm{d}y$$

If seeing $1 - x^2$, then substitute $x = \sin y$



Example: Machine translation

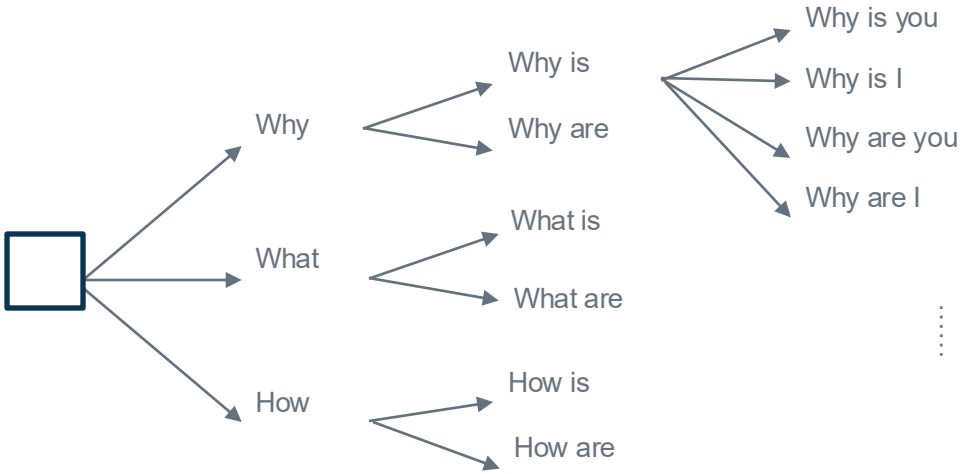
Translate “你好嗎” to English

States: current word sequence

Actions: the next word

Successor: the concatenation of current sequence and next word

Goal test: whether the current sequence means the same as 你好嗎



Example: Machine translation

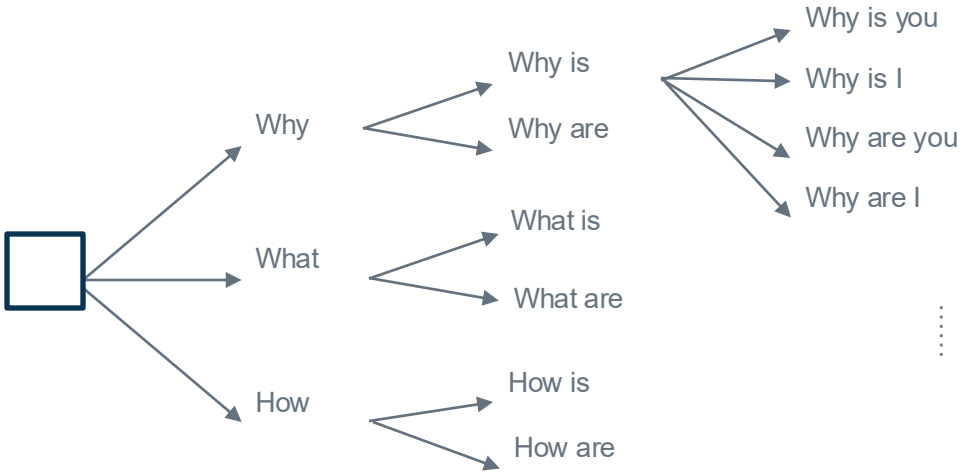
Translate “你好嗎” to English

States: current word sequence

Actions: the next word

Successor: the concatenation of current sequence and next word

Goal test: whether the current sequence means the same as 你好嗎

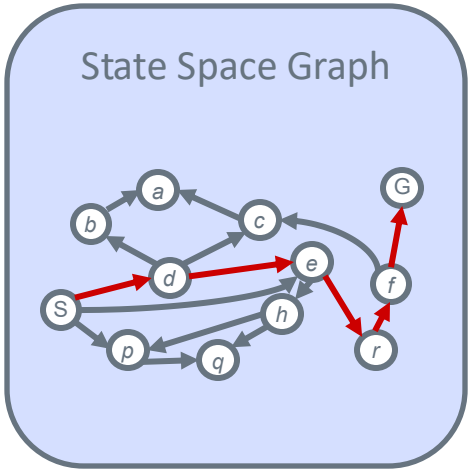


Topics

- BFS
- DFS
- UCS (Dijkstra Algorithm)
- Difference with DSA2:
  - The state space is exponentially large, and it’s unlikely we’ll store the whole state space in memory

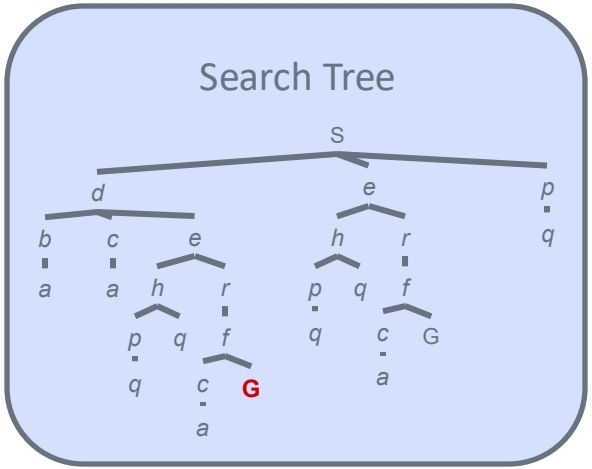
General Framework

State Space and Search Tree



Each NODE in in the search tree is an entire PATH in the state space graph.

We construct both on demand – and we construct as little as possible.



A General Framework

Expanded ← { }

Frontier ← { initial_state }

While **Frontier** is not empty:

Choose a node s from **Frontier** ①

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If s' has not been reached

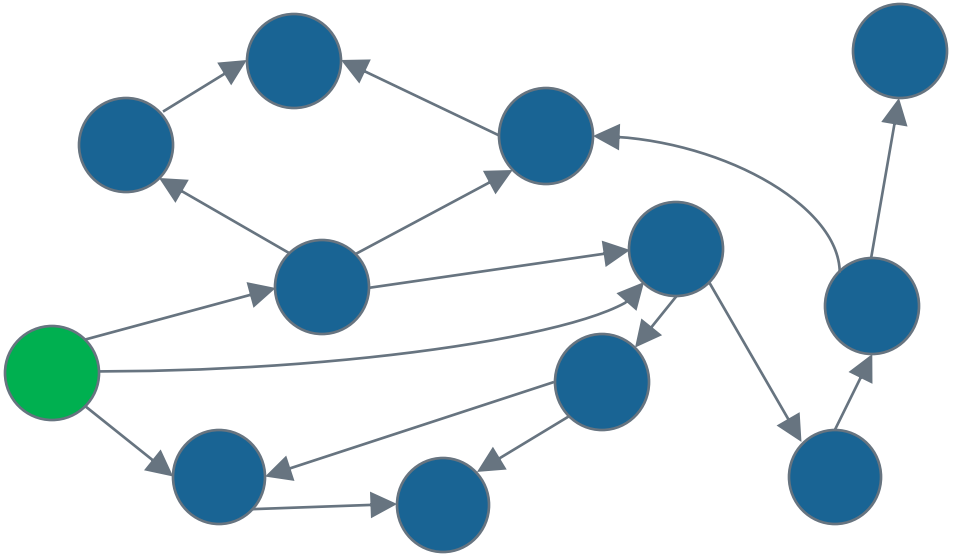
Put s' in **Frontier** ②

Move s to **Expanded** ③

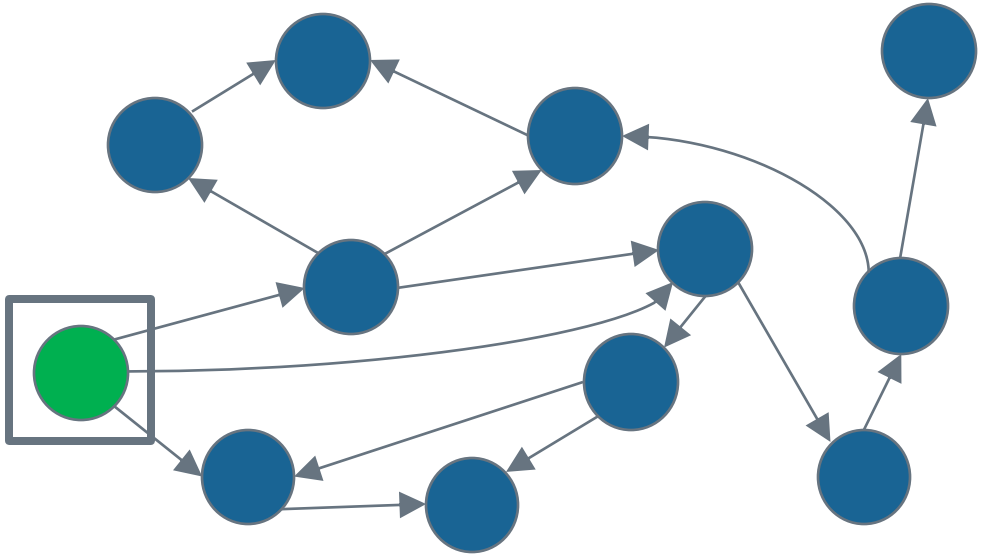
All nodes that are not in **Expanded** or **Frontier**

Nodes are divided into 3 groups: **Expanded**, **Frontier**, and **Unreached.**

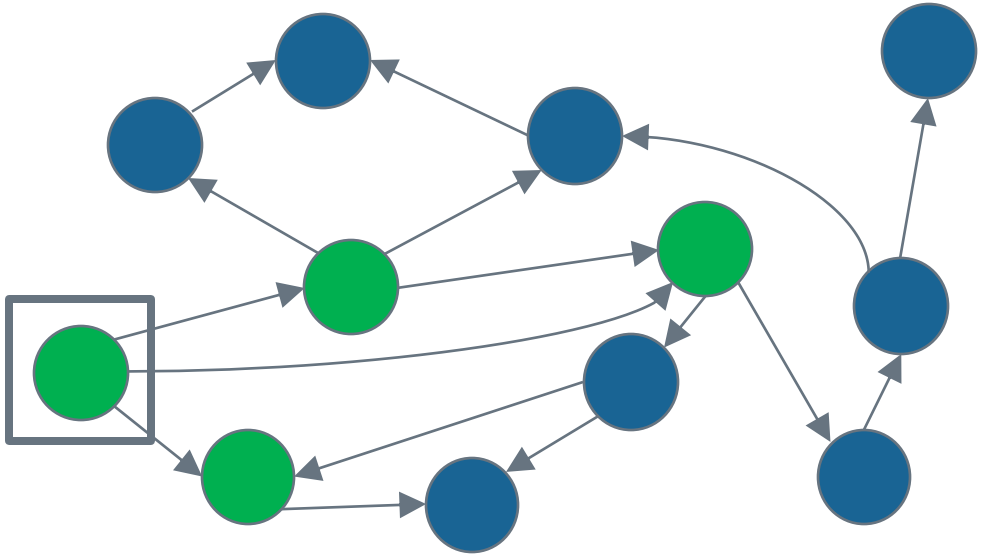
⓪ Place the initial state in **Frontier**



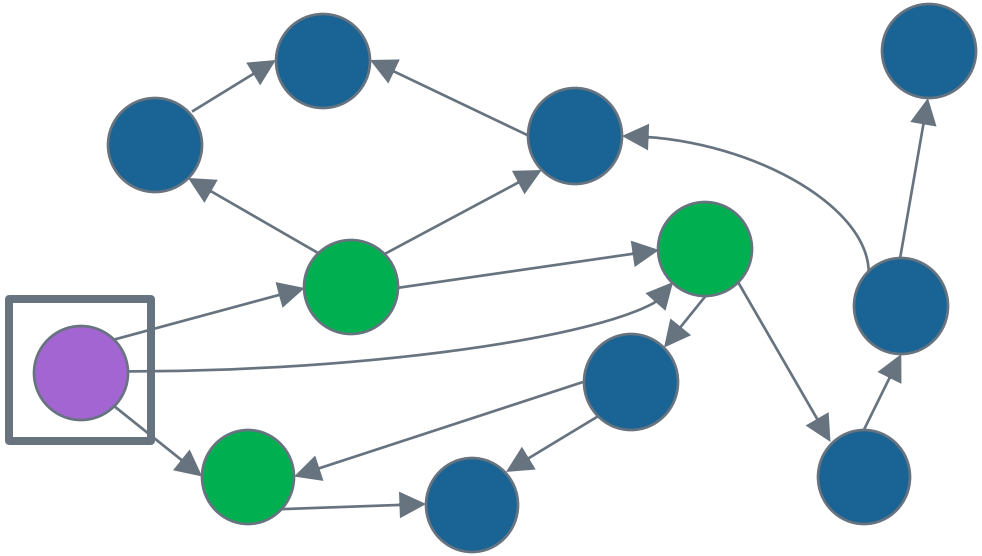
① Choose a node s from **Frontier**



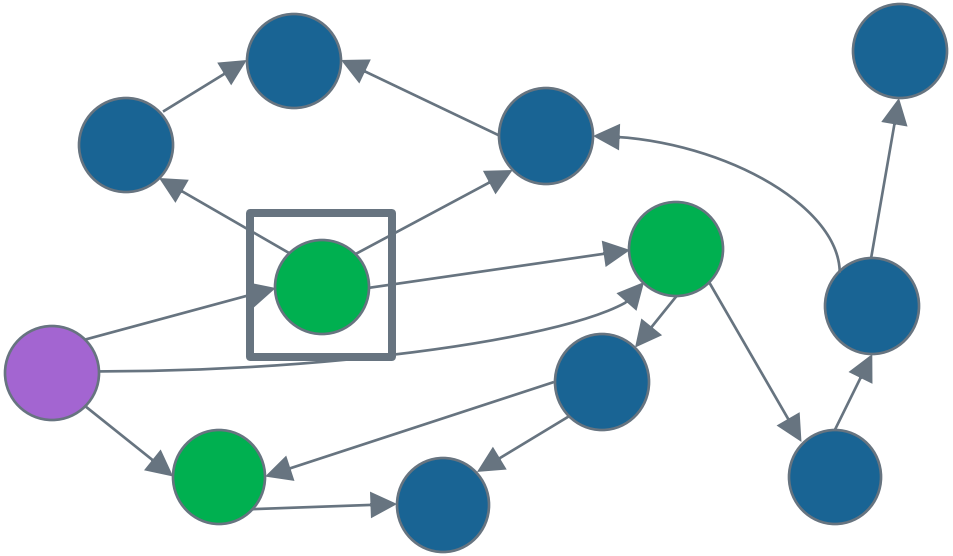
② Move all unreached successors of s to **Frontier**



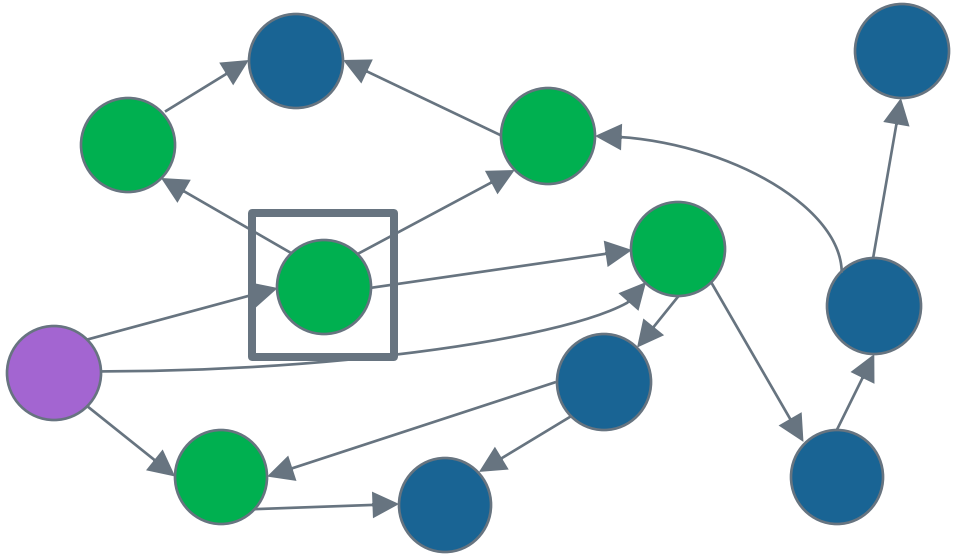
③ Move s to **Expanded**



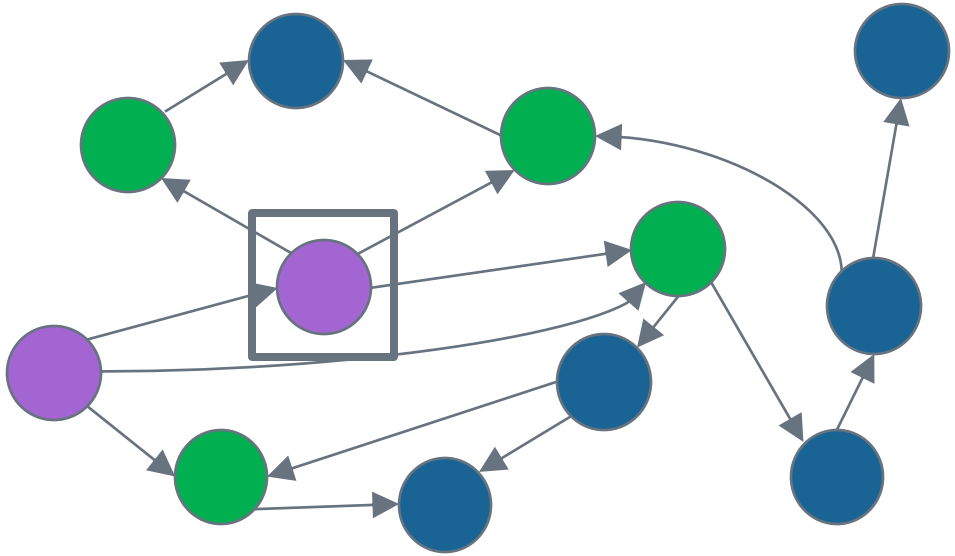
① Choose a node s from **Frontier**



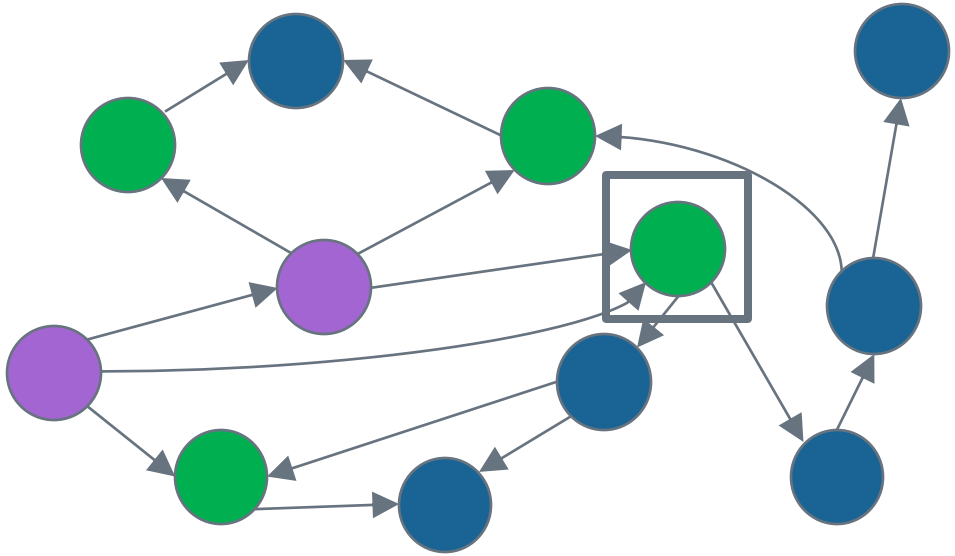
② Move all unreached successors of s to **Frontier**



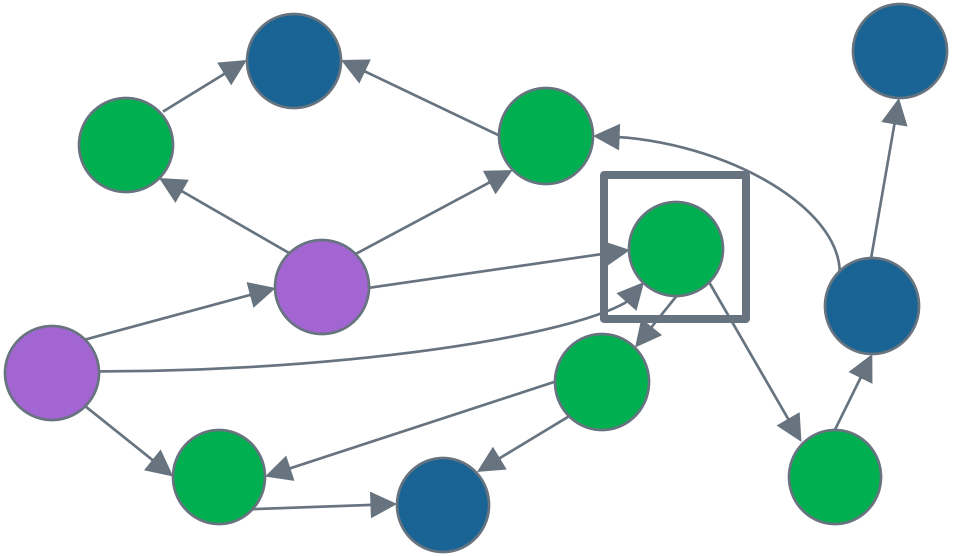
③ Move s to **Expanded**



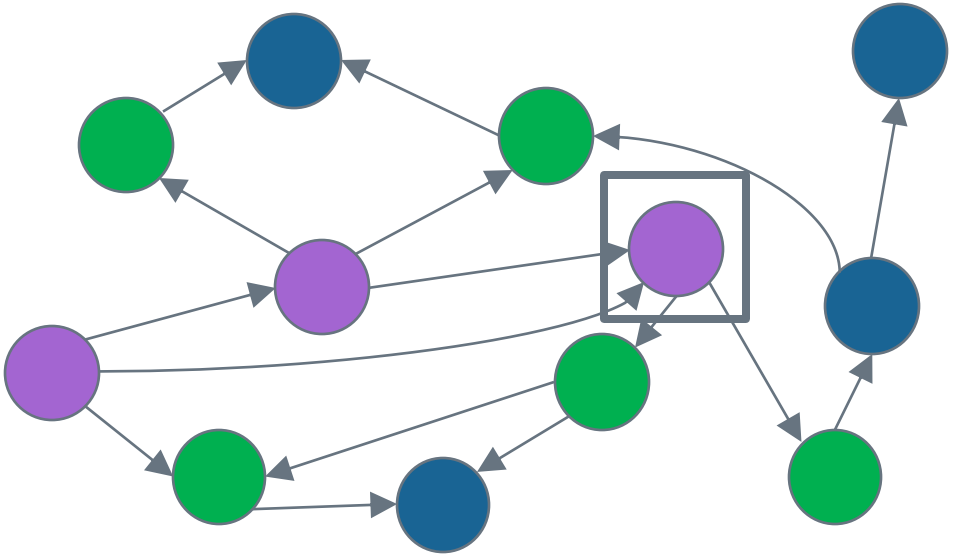
① Choose a node s from **Frontier**



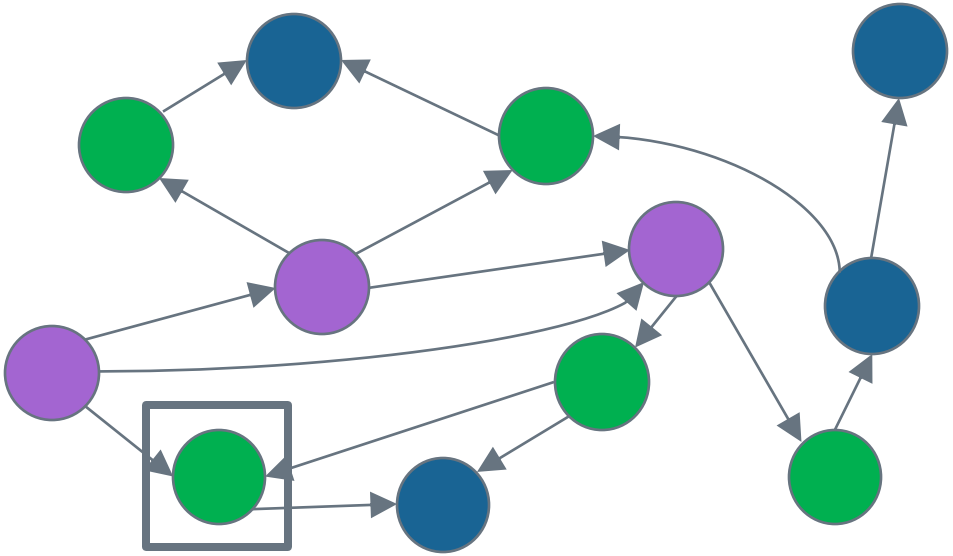
② Move all unreached successors of s to **Frontier**



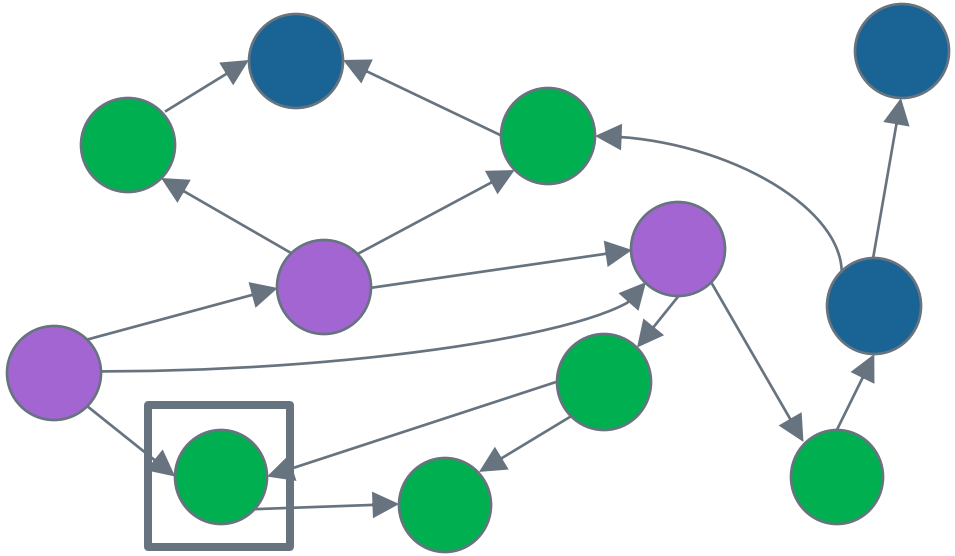
③ Move s to **Expanded**



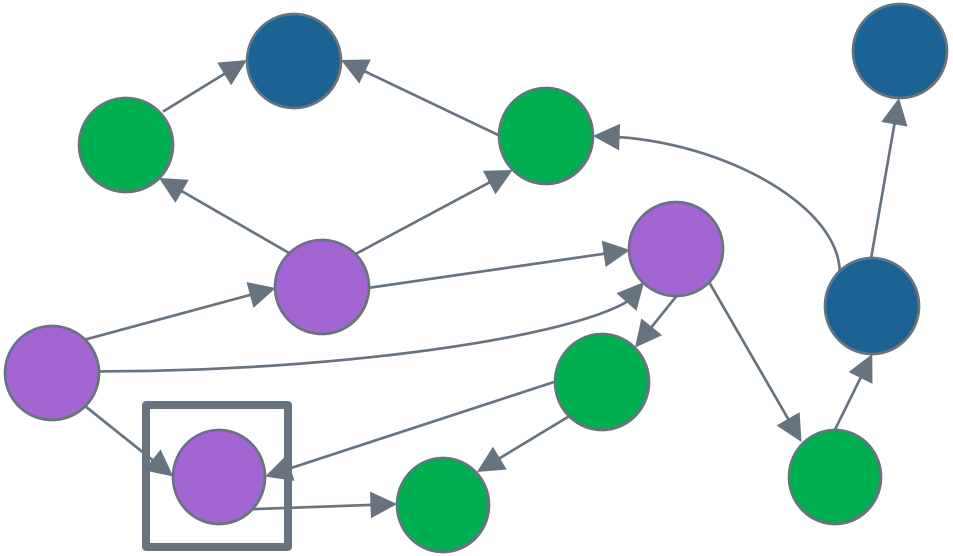
① Choose a node s from **Frontier**



② Move all unreached successors of s to **Frontier**



③ Move s to **Expanded**



Implementation

Expanded ← { }

Frontier ← { initial_state }

While **Frontier** is not empty:

Choose a node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If s' has not been reached:

Put s' in **Frontier**

Move s to **Expanded**

Choose and then remove it

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If not **Reached**[s']:

Put s' in **Frontier**

Reached[s'] ← True

Termination When Goal is Encountered

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

If s is a goal state, terminate

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If not Reached[s']:

Put s' in **Frontier**

Reached[s'] ← True

Late Goal Test

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If not Reached[s']:

If s' is a goal state, terminate

Push s' to **Frontier**

Reached[s'] ← True

Early Goal Test

Termination When Goal is Encountered

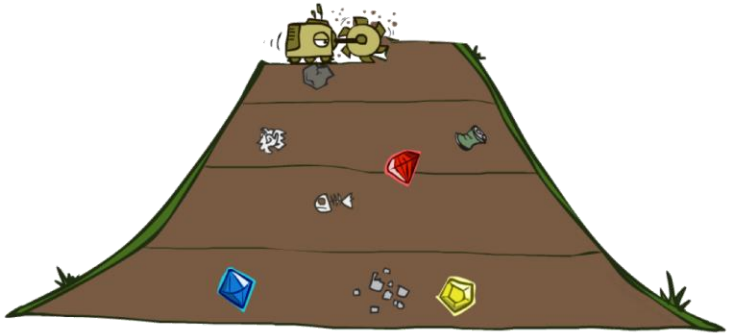
- Early Goal Test allows quicker termination when a goal is found.
  - Breadth First Search
  - Depth First Search

- However, when actions are associated with costs and we want to find a **minimum cost solution** (i.e., cost-sensitive), we may have to use the Late Goal Test.
  - Uniform Cost Search (Dijkstra Algorithm)

Uninformed Search

DFS and BFS

Depth First Search



Breadth First Search

DFS and BFS

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

differ here

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If not Reached[s']:

If s' is a goal state, terminate

Push s' to **Frontier**

Reached[s'] ← True

DFS

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop the **newest** node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

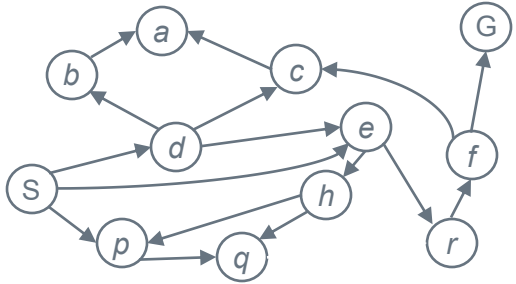
If not Reached[s']:

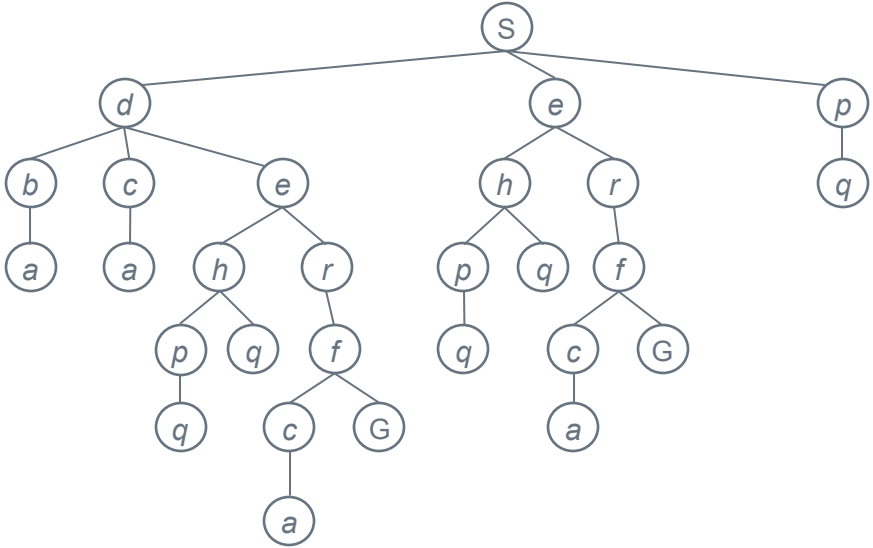
If s' is a goal state, terminate

Push s' to **Frontier**

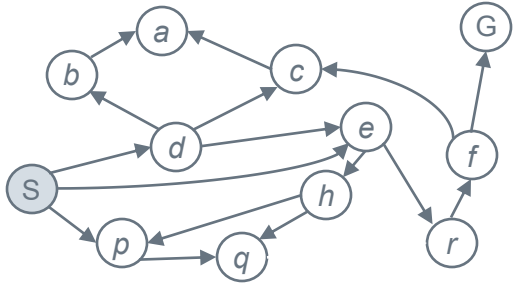
Reached[s'] ← True

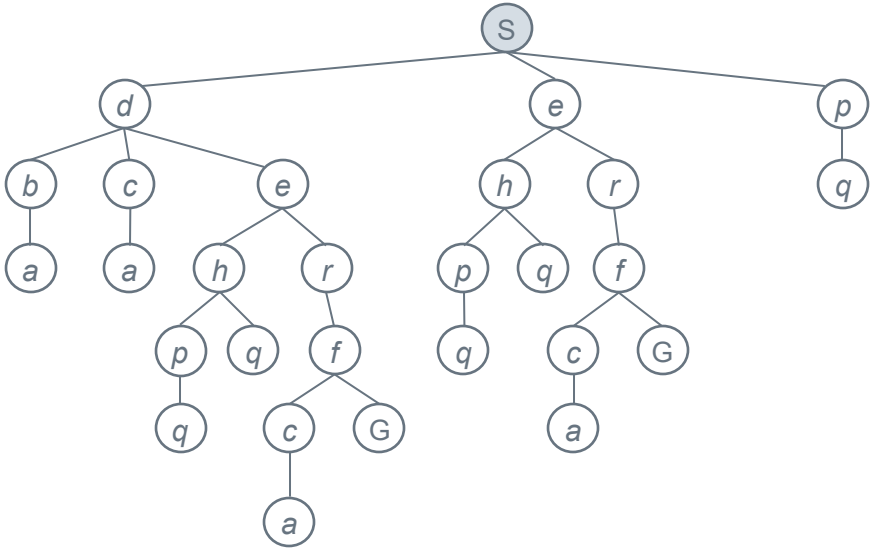
DFS



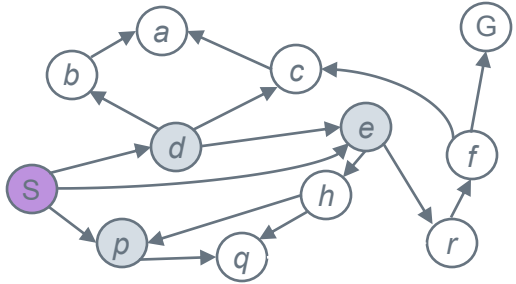


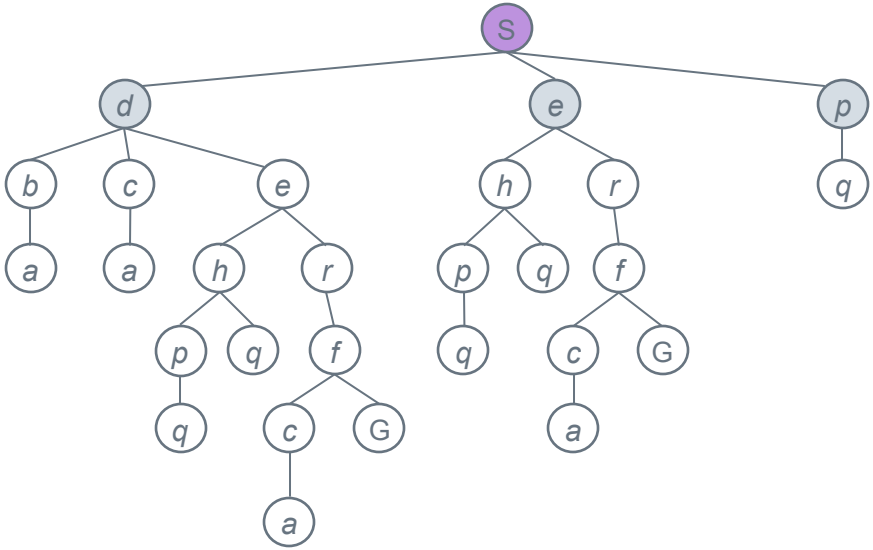
DFS



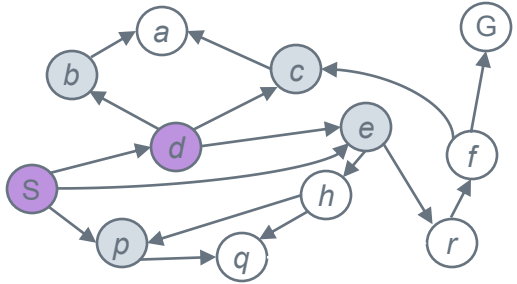


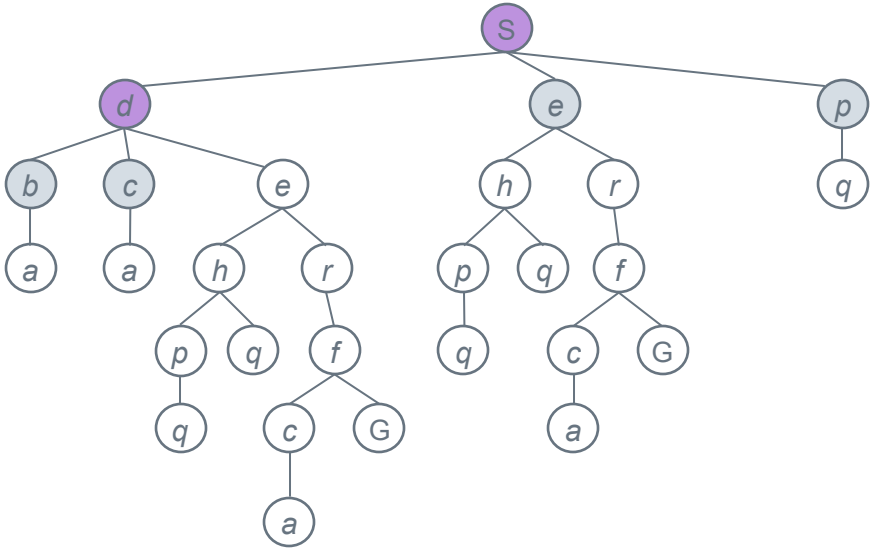
DFS



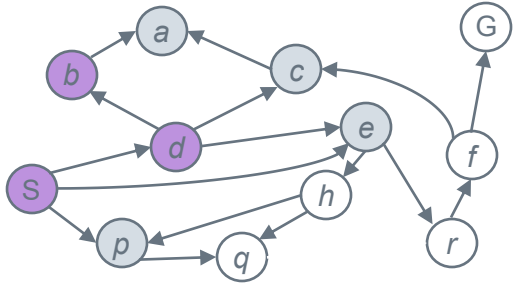


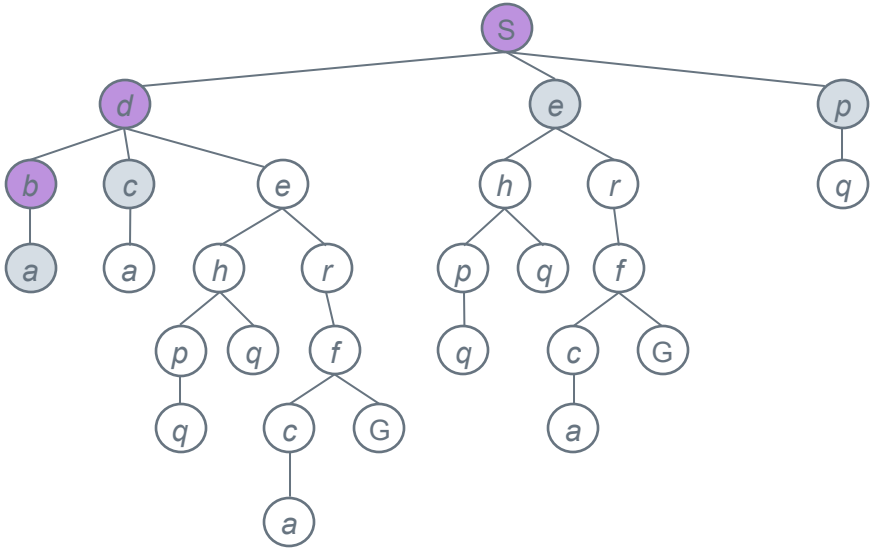
DFS



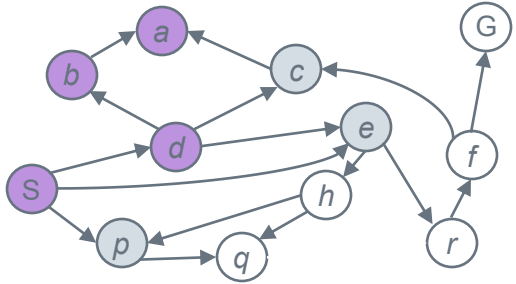


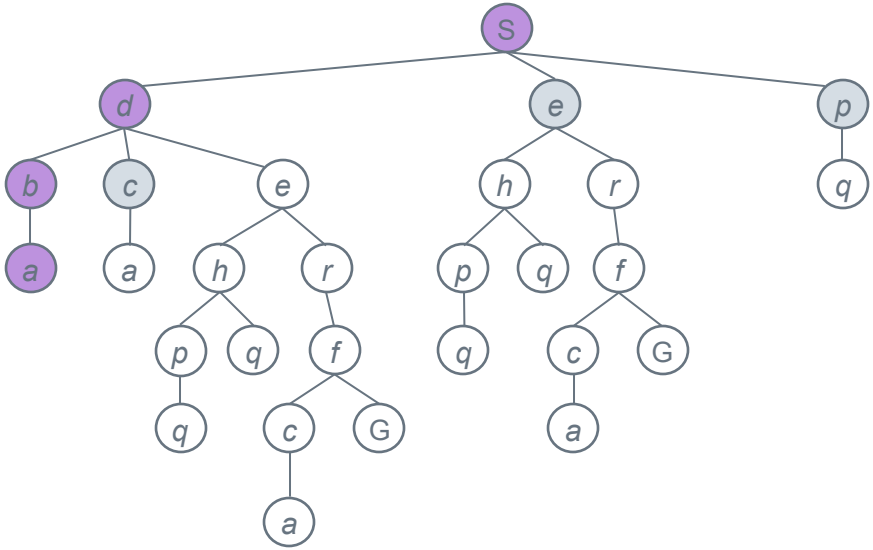
DFS



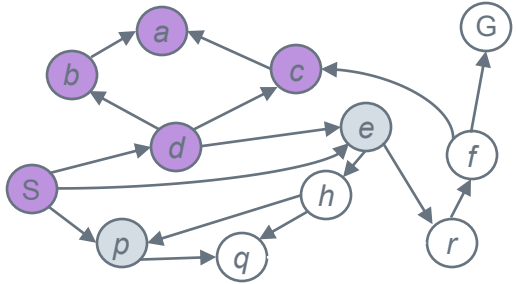


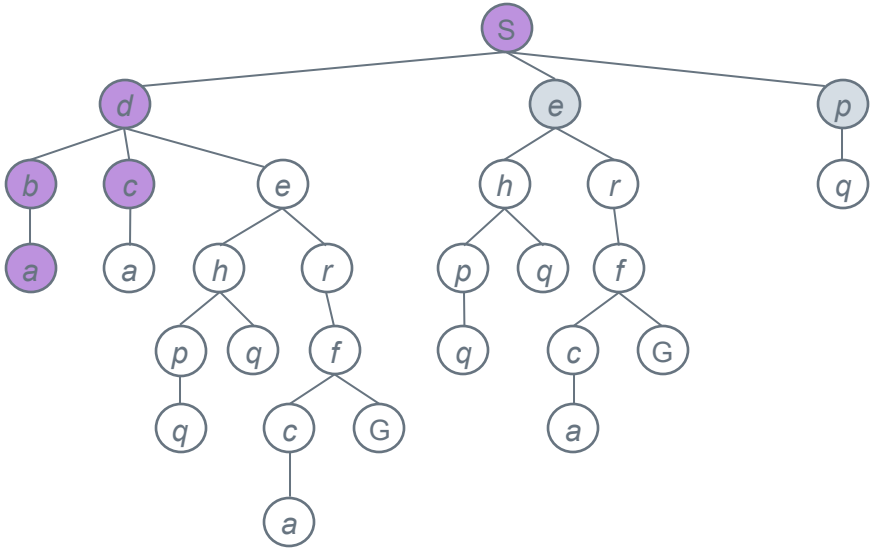
DFS



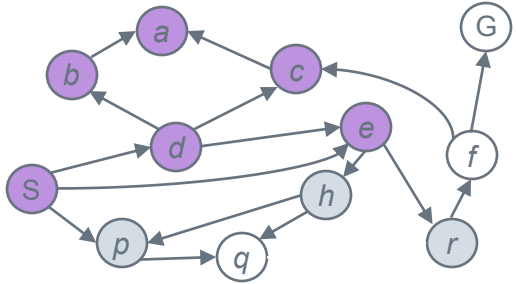


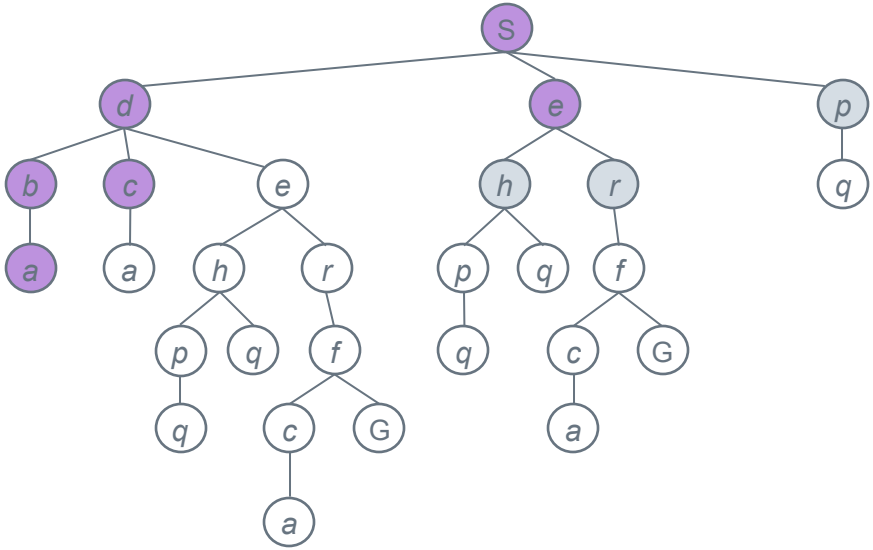
DFS



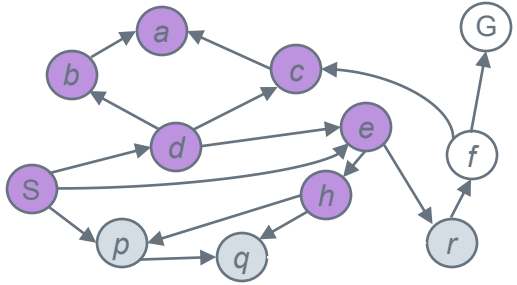


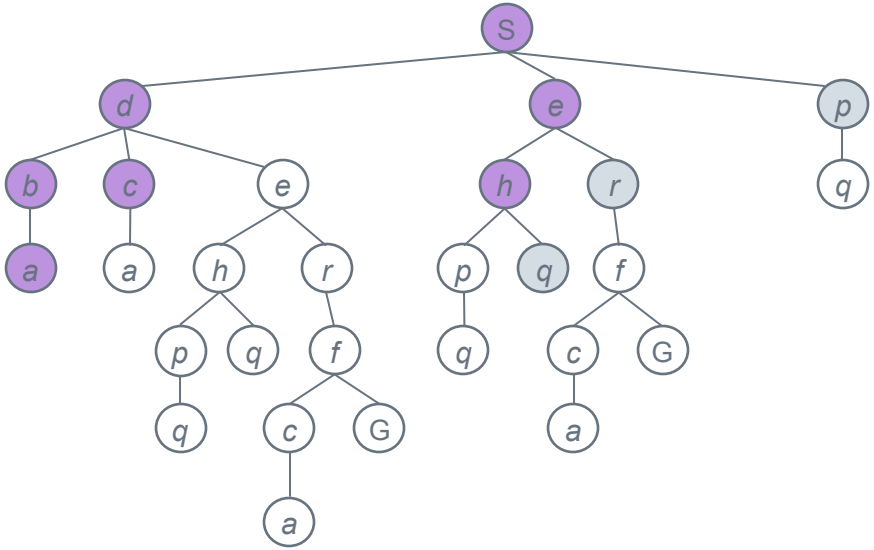
DFS



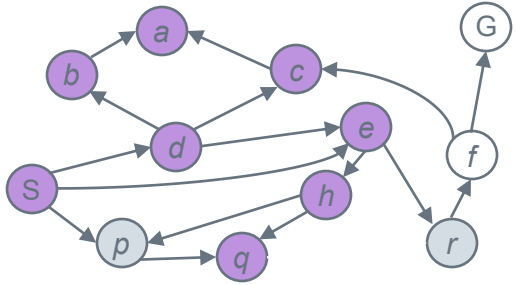


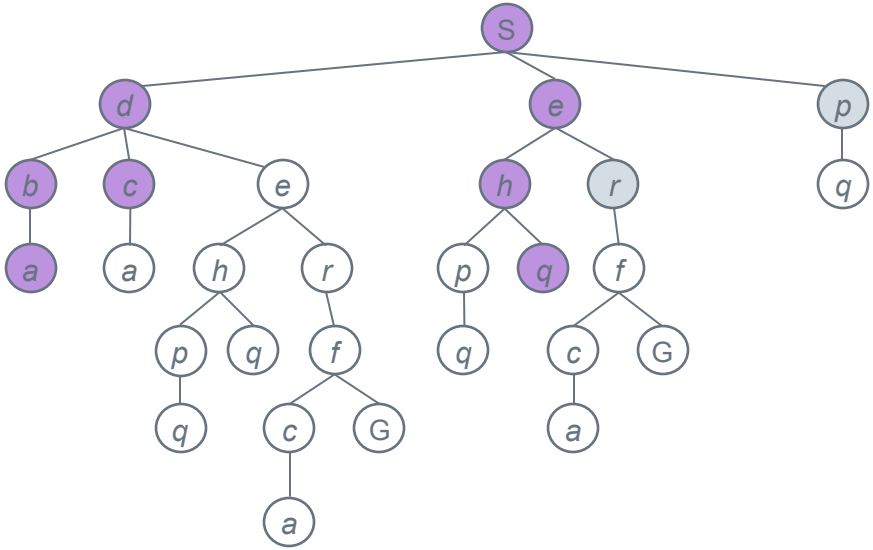
DFS



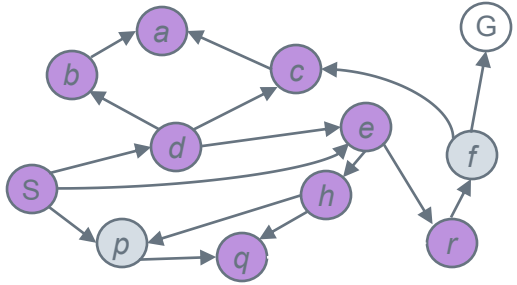


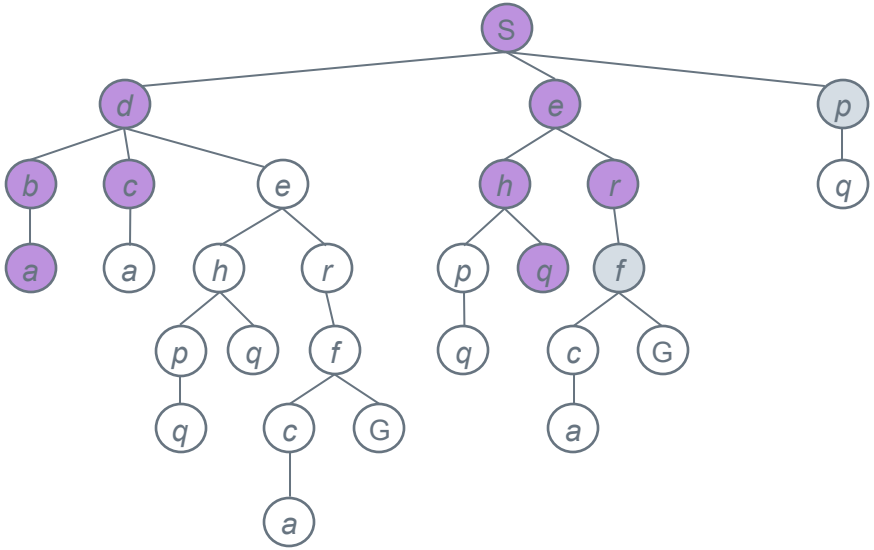
DFS



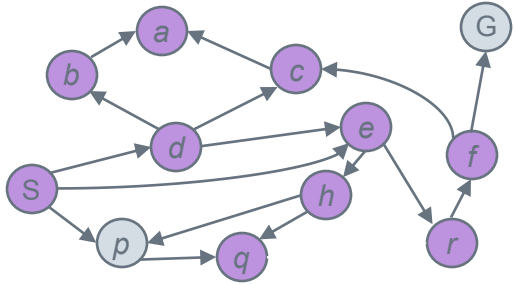


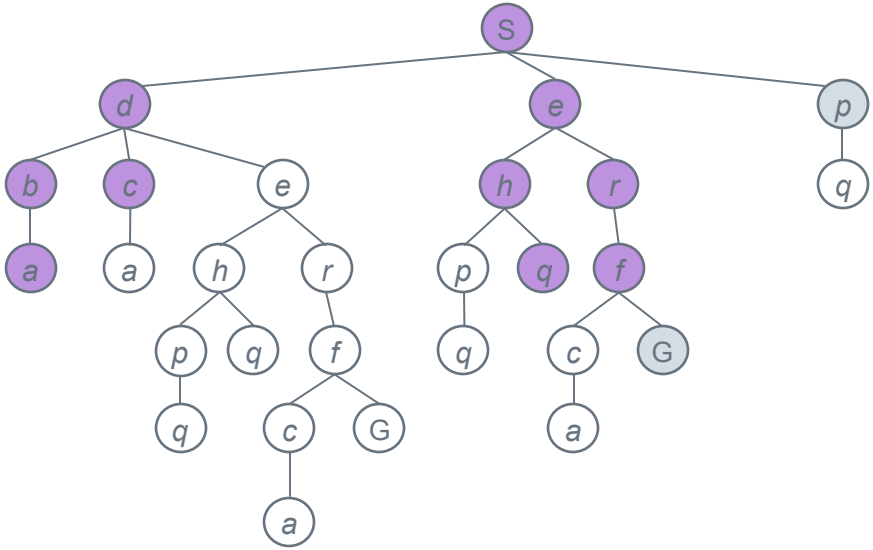
DFS





DFS





BFS

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop the **oldest** node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

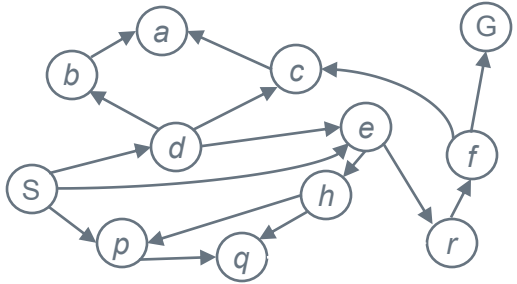
If not Reached[s']:

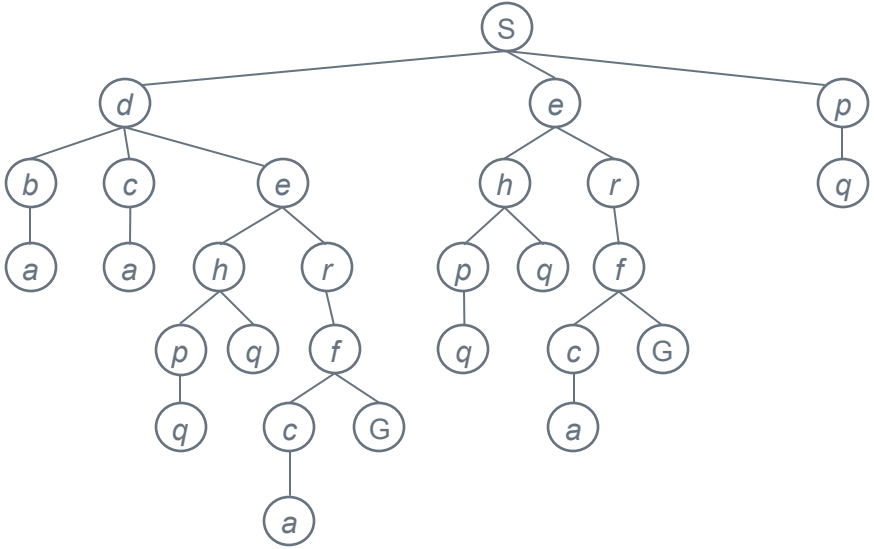
If s' is a goal state, terminate

Push s' to **Frontier**

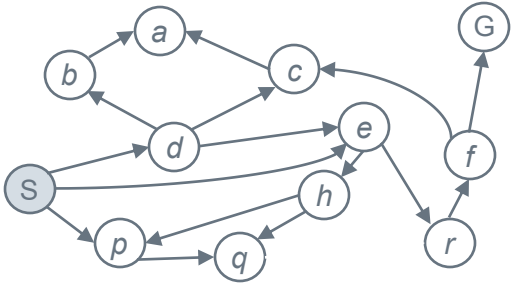
Reached[s'] ← True

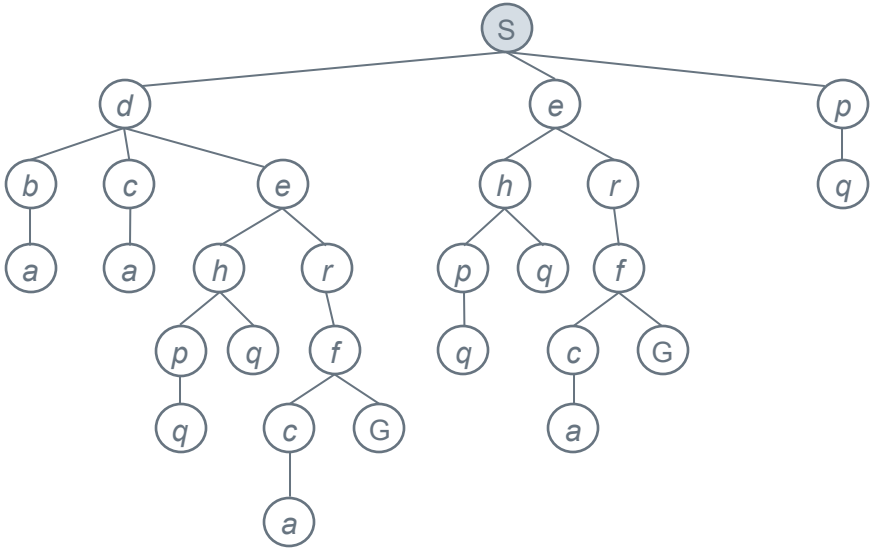
BFS



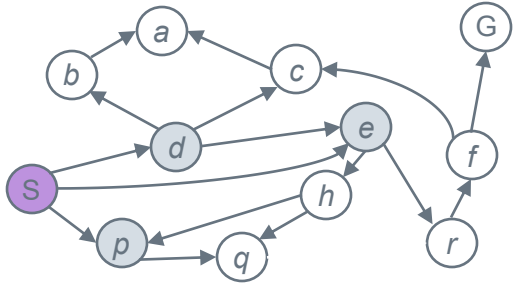


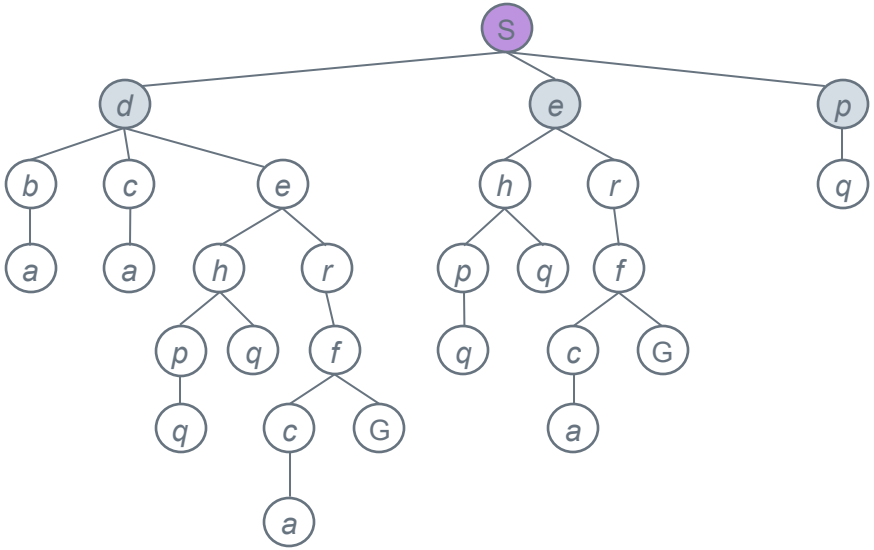
BFS



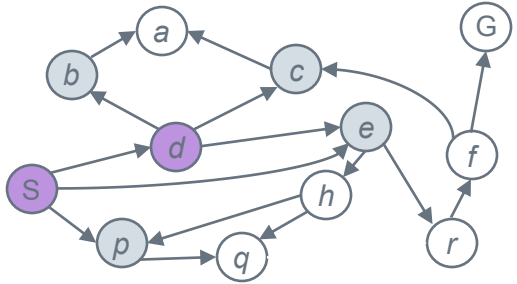


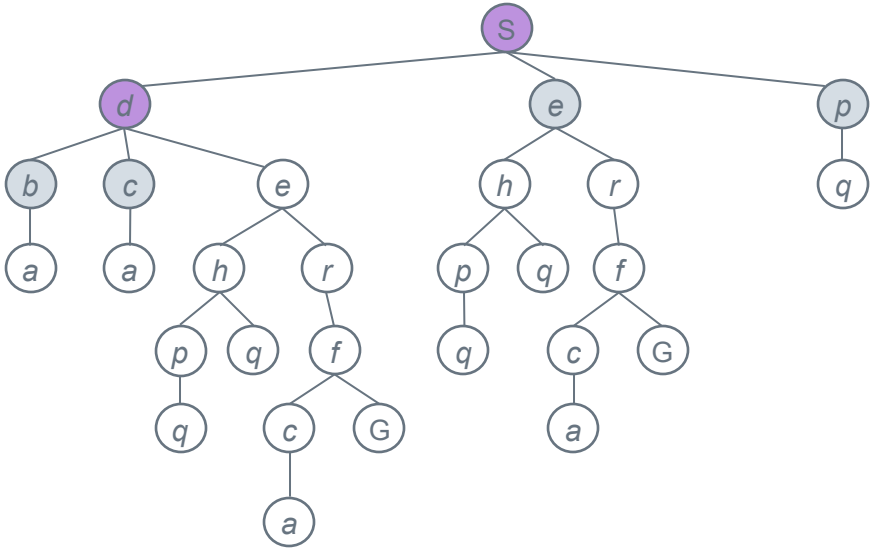
BFS



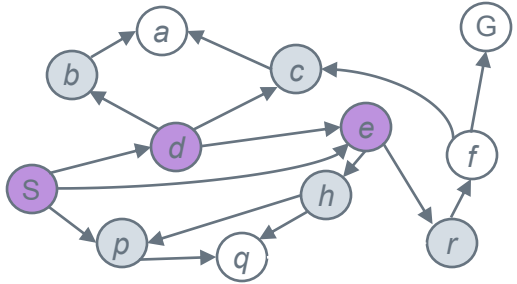


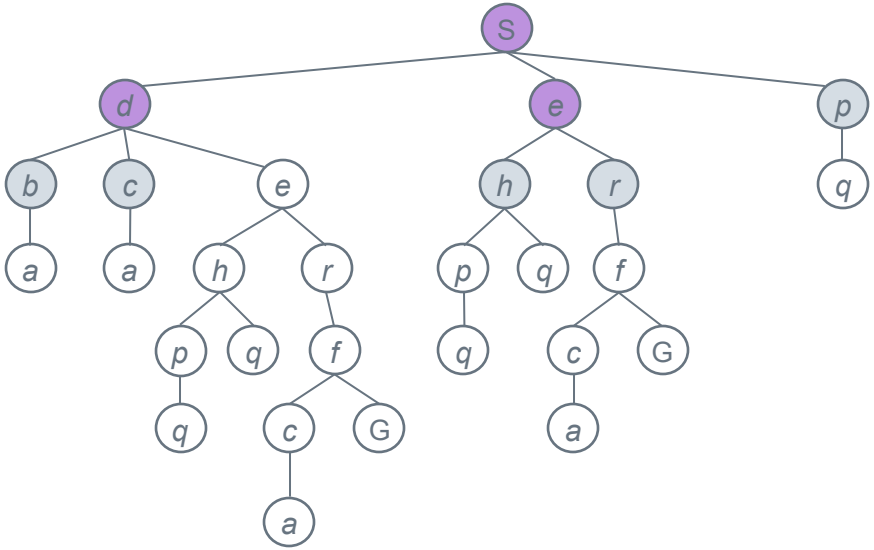
BFS



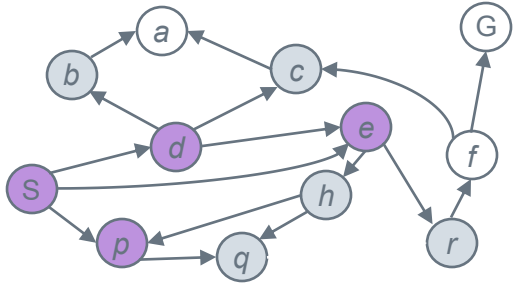


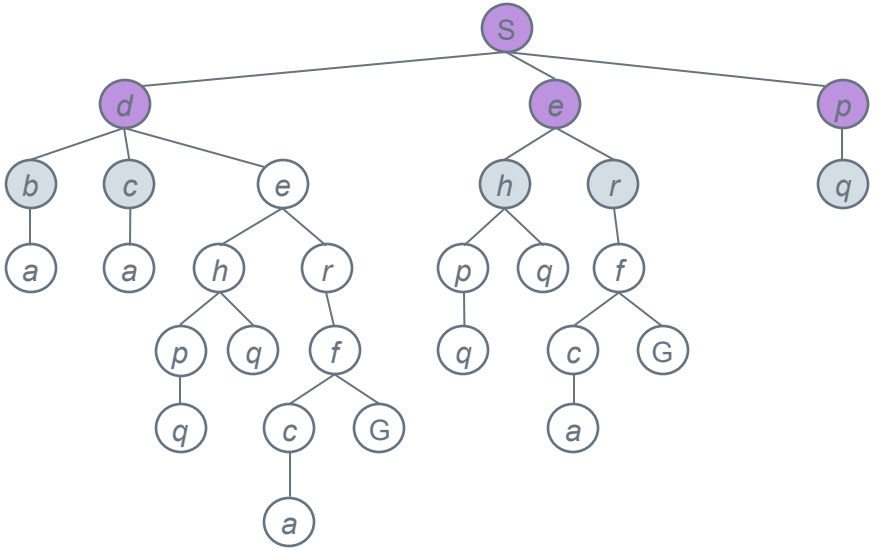
BFS



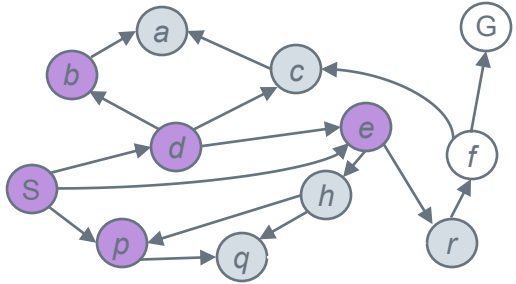


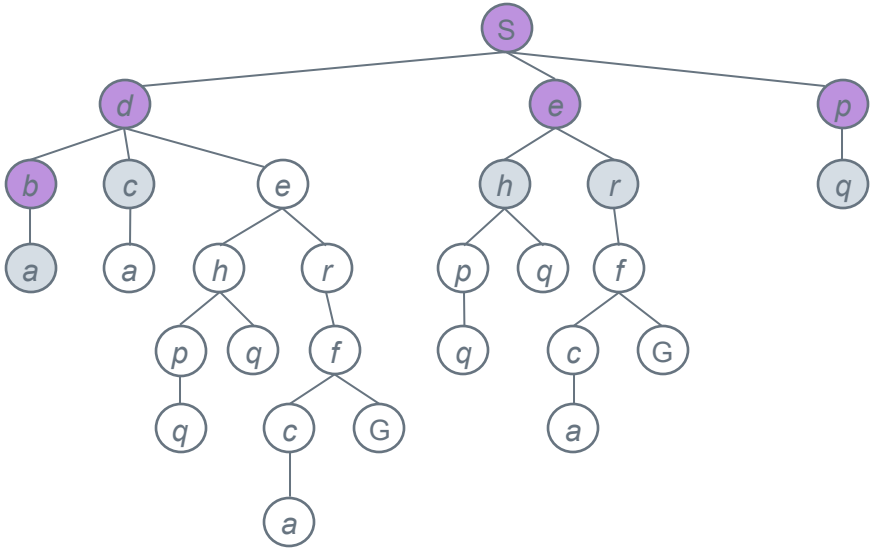
BFS



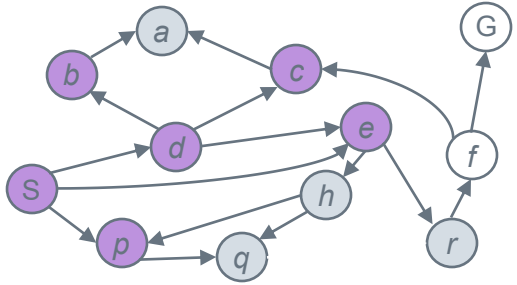


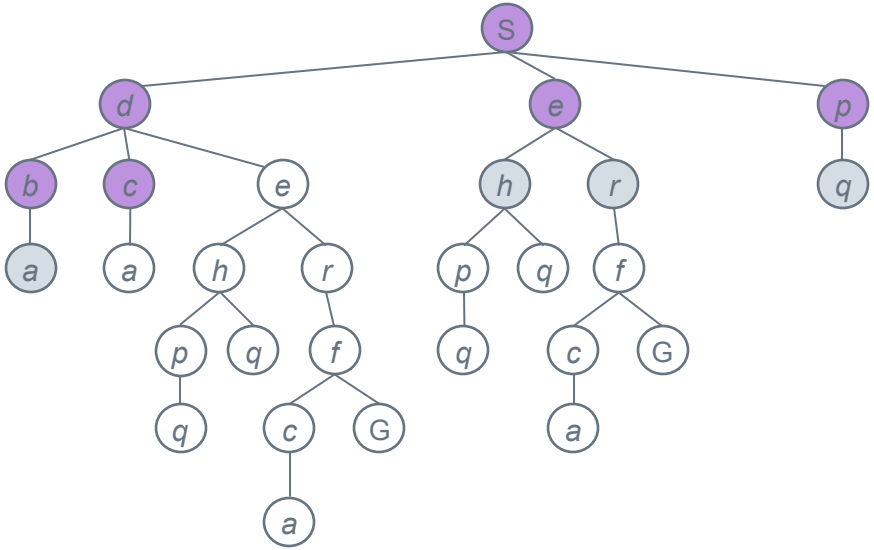
BFS



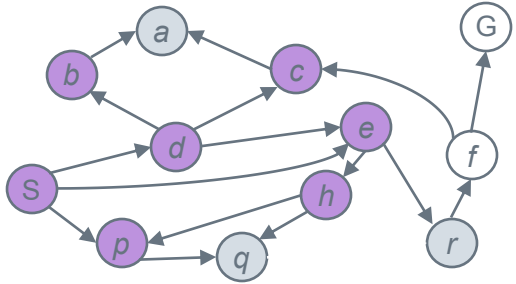


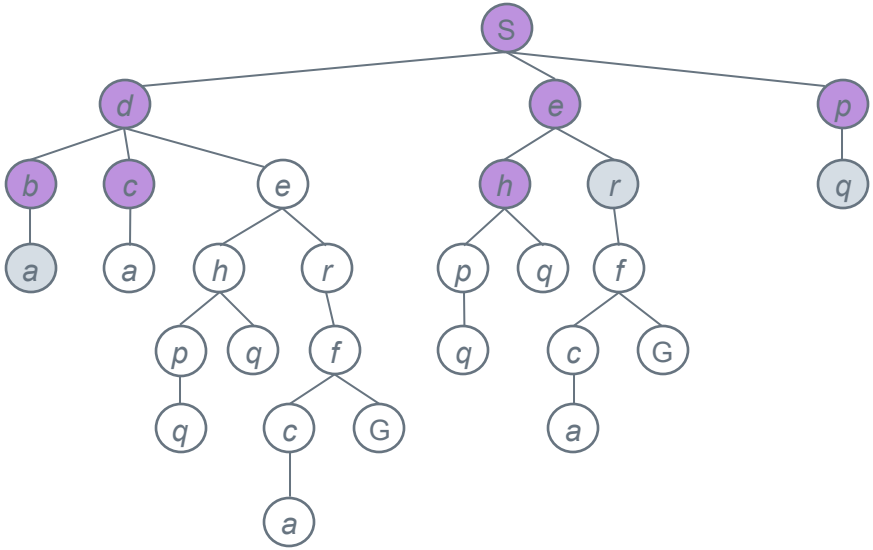
BFS



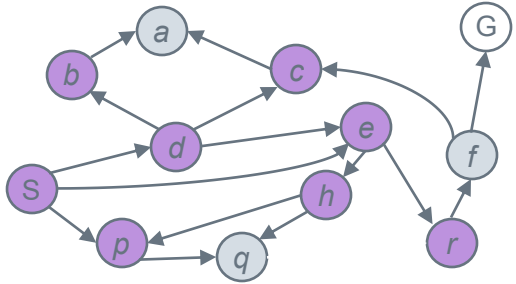


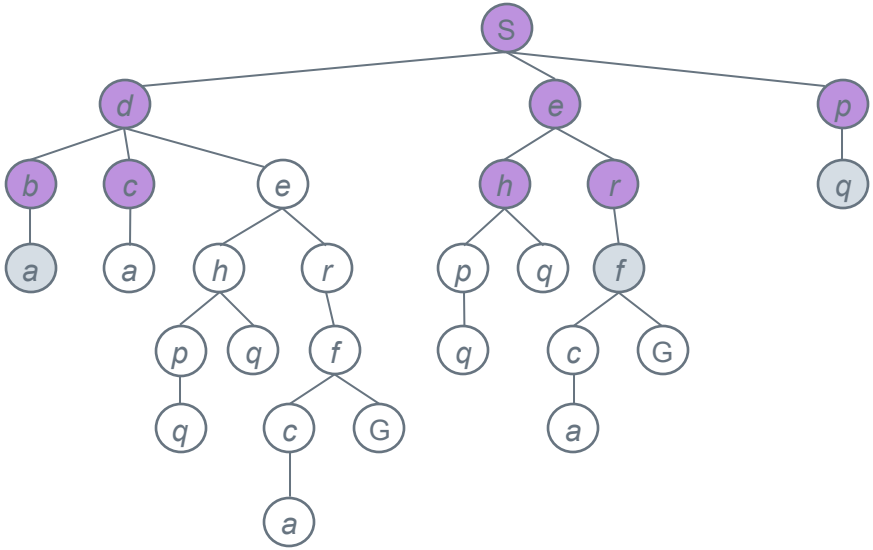
BFS



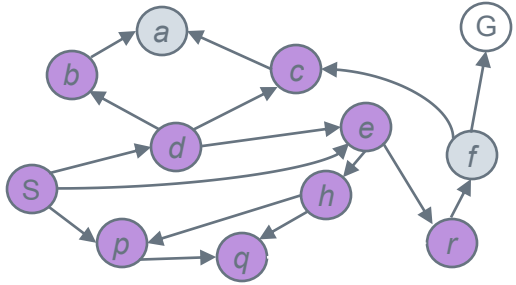


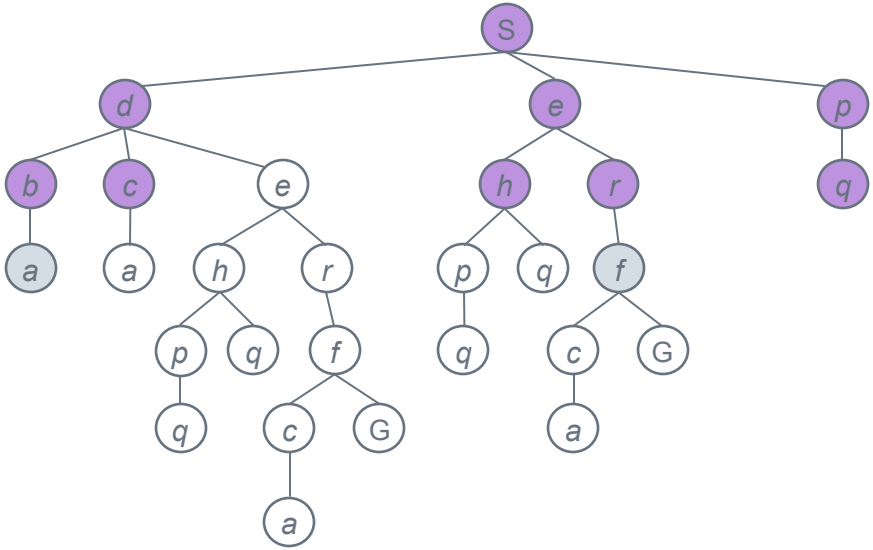
BFS



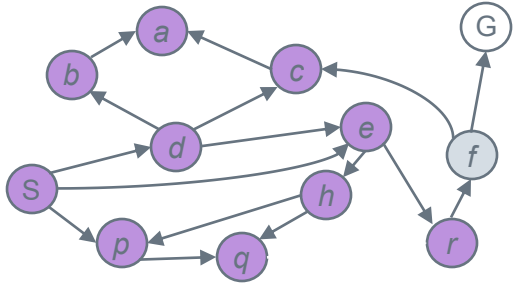


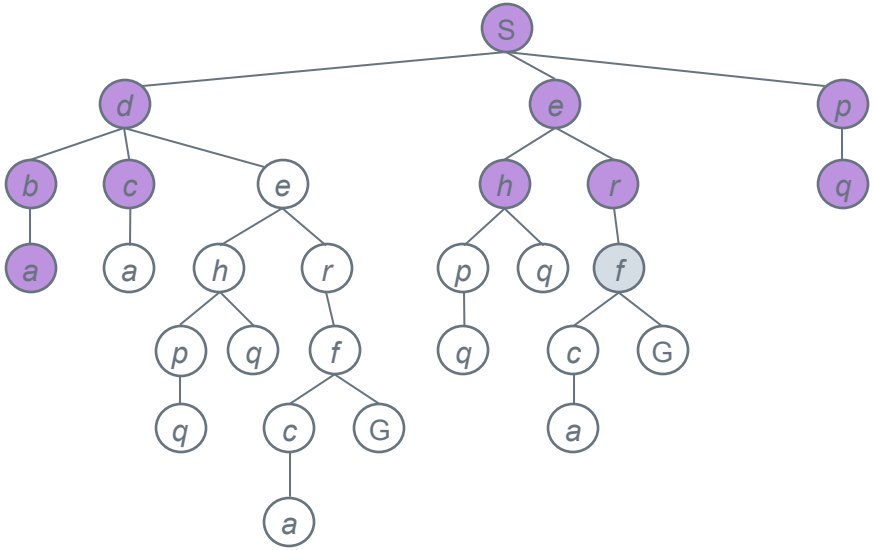
BFS



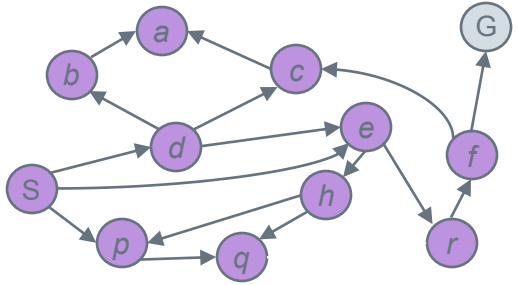


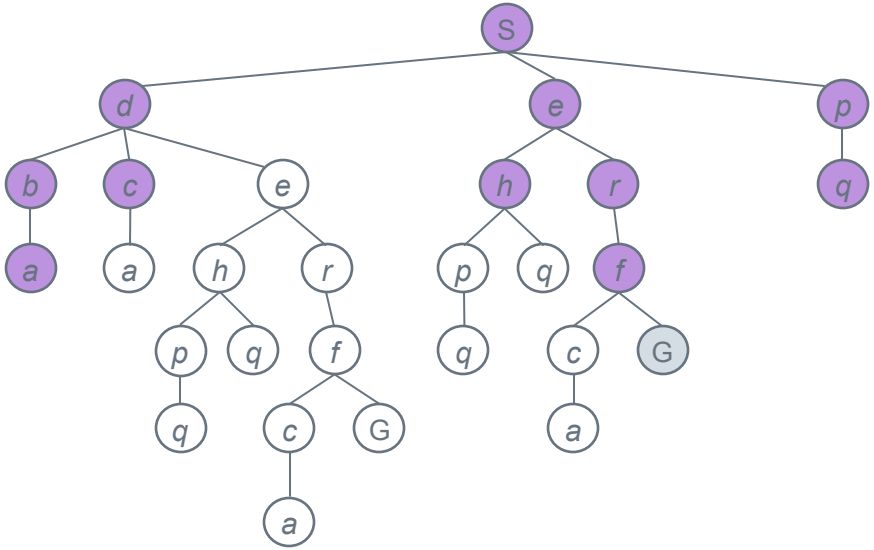
BFS





BFS





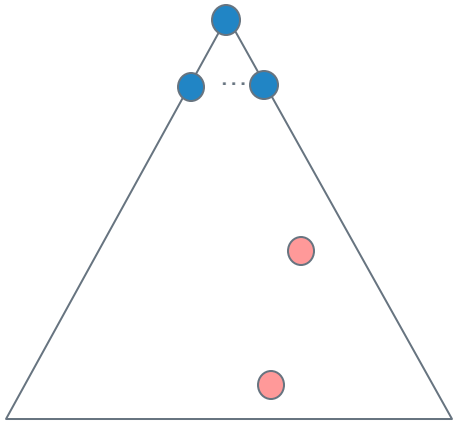
DFS vs. BFS

In what cases is DFS quicker to find the goal?

In what cases is BFS quicker to find the goal?

DFS vs. BFS

Does DFS / BFS find the goal with the smallest depth?

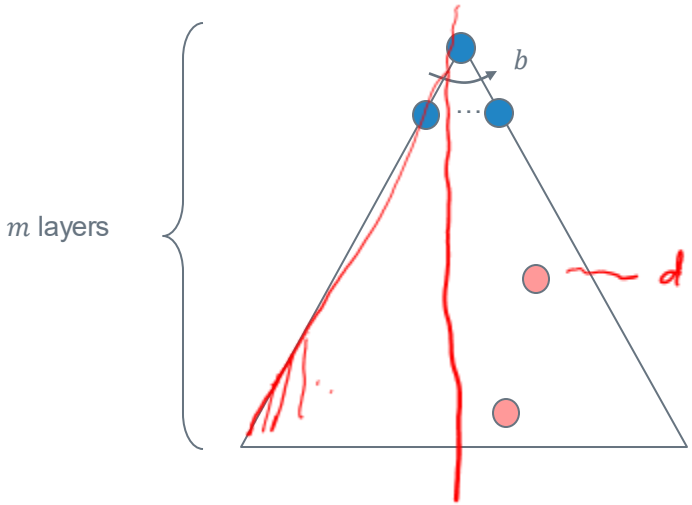


DFS vs. BFS

Suppose there exists a goal at layer $\leq d$. What is the time complexity for DFS / BFS to find a goal?

BFS: $O(\underbrace{b}_{\text{expand layer 1}} + b^2 + \cdots + b^{d-1}) = O(b^d)$

DFS: $O(b^m)$



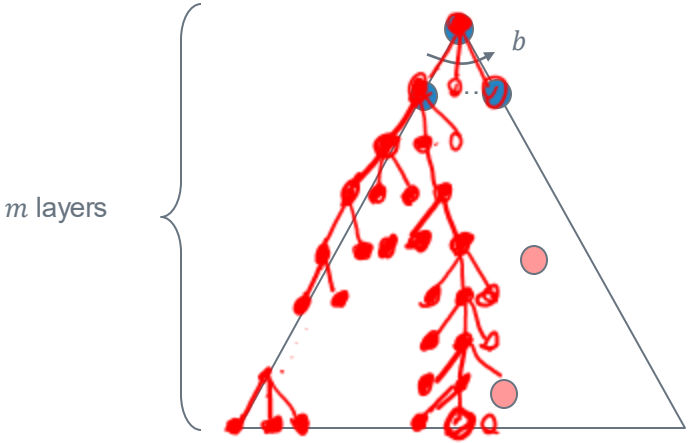
b: branching factor
m: maximum depth
Goals at various depths

DFS vs. BFS

What's the maximum possible size of **Frontier** in DFS / BFS?

BFS: $O(b^d)$

DFS: $O(bm)$



b: branching factor
m: maximum depth
Goals at various depths

$m > d$

DFS vs. BFS

	Time	Frontier Size
DFS	b^m	bm
BFS	b^d	b^d

So DFS can be more memory-efficient than BFS?

Yes … but not with our current implementation

DFS

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop the newest node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If not **Reached**[s']:

If s' is a goal state, terminate

Push s' to **Frontier**

Reached[s'] ← True

DFS

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop the newest node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

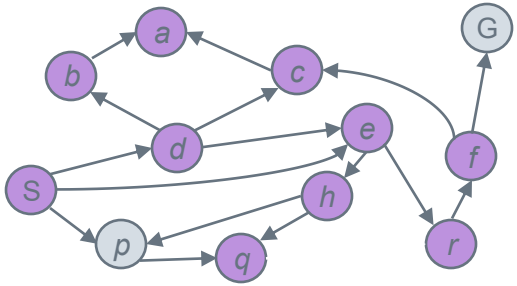
If s' is a goal state, terminate

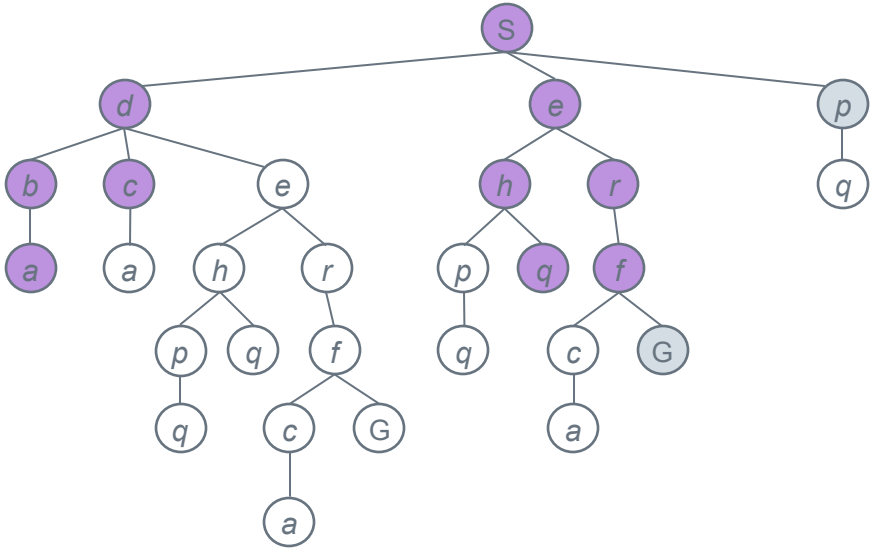
Push s' to **Frontier**

Because we omit the check, the algorithm may end up search in same sub-trees multiple times.

A Memory Efficient Version of DFS for Acyclic Graphs

Previous DFS Example





DFS

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop the **newest** node s from **Frontier**

For all action a:

$s' \leftarrow \mathrm{succ}(s, a)$

If s' is not an ancestor of s:

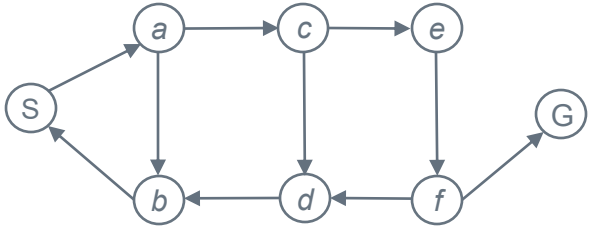
If s' is a goal state, terminate

Push s' to **Frontier**

A Memory Efficient Version of DFS for Cyclic Graphs

handling cycles

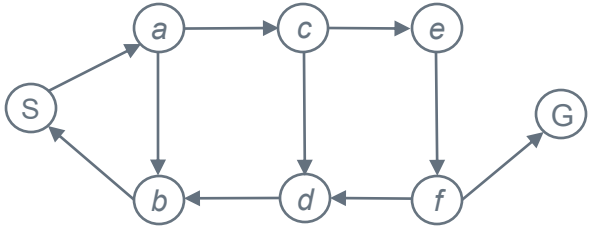
(Memory Efficient) DFS



s

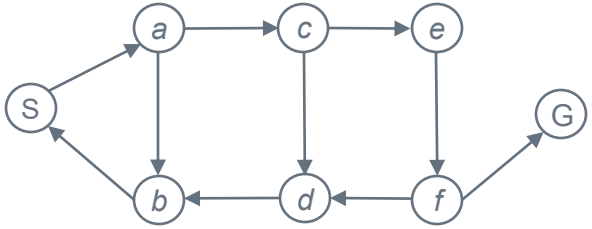
handling cycles

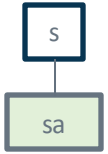
(Memory Efficient) DFS



handling cycles

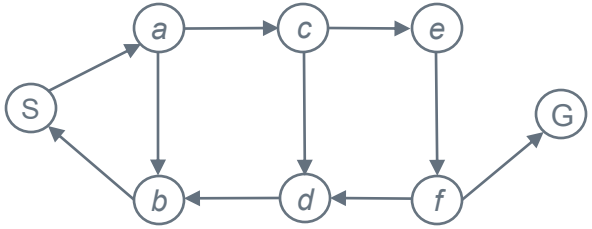
(Memory Efficient) DFS

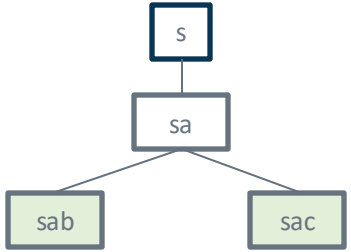




handling cycles

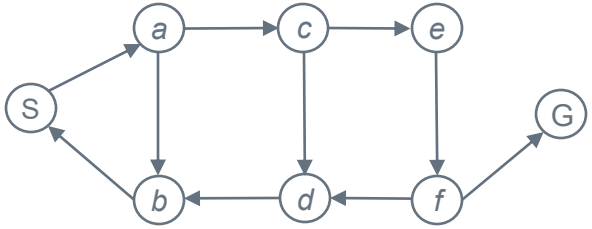
(Memory Efficient) DFS

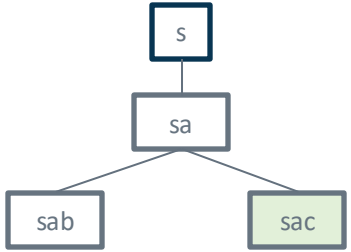




handling cycles

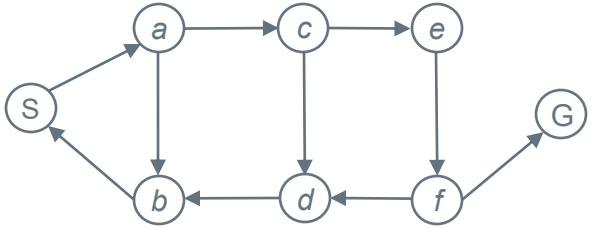
(Memory Efficient) DFS

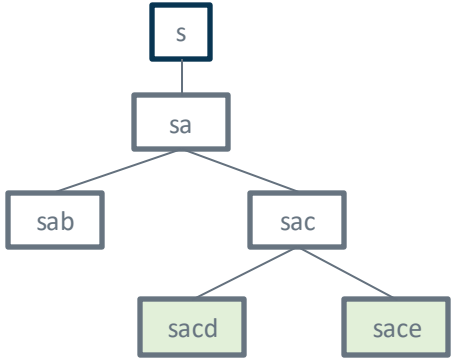




handling cycles

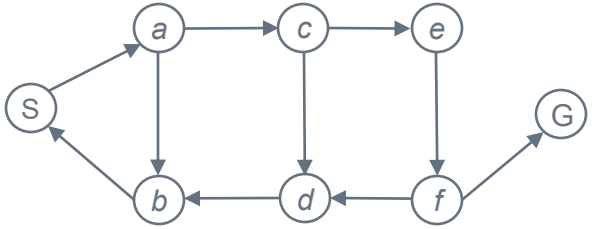
(Memory Efficient) DFS

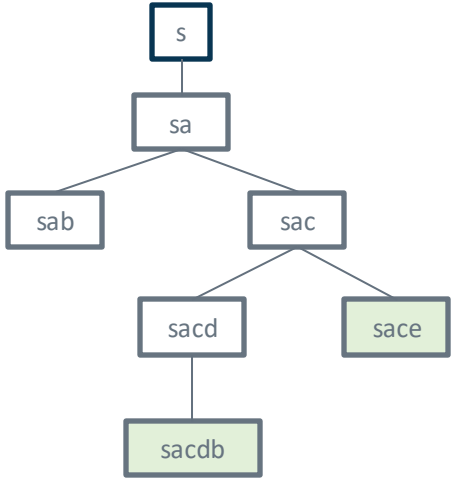




handling cycles

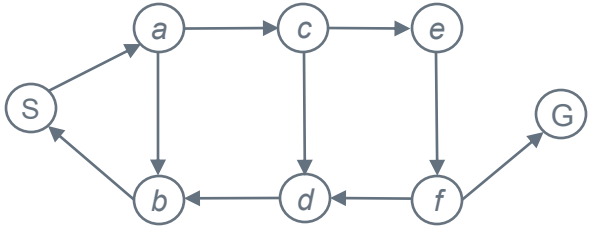
(Memory Efficient) DFS

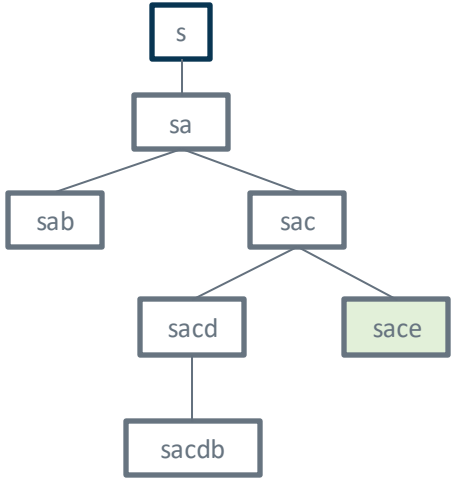




handling cycles

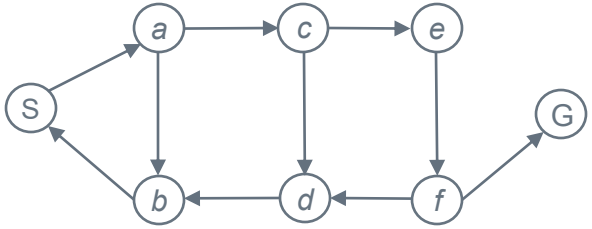
(Memory Efficient) DFS

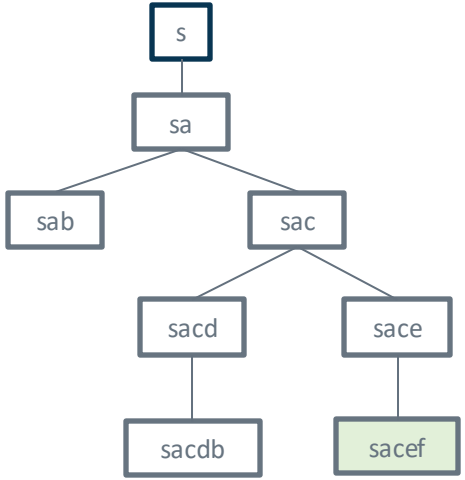




handling cycles

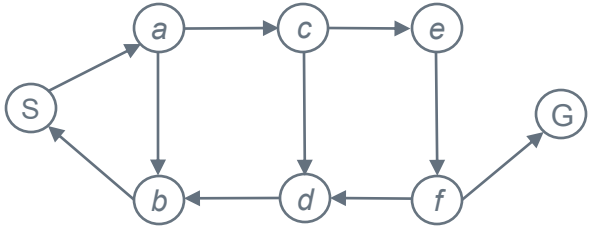
(Memory Efficient) DFS

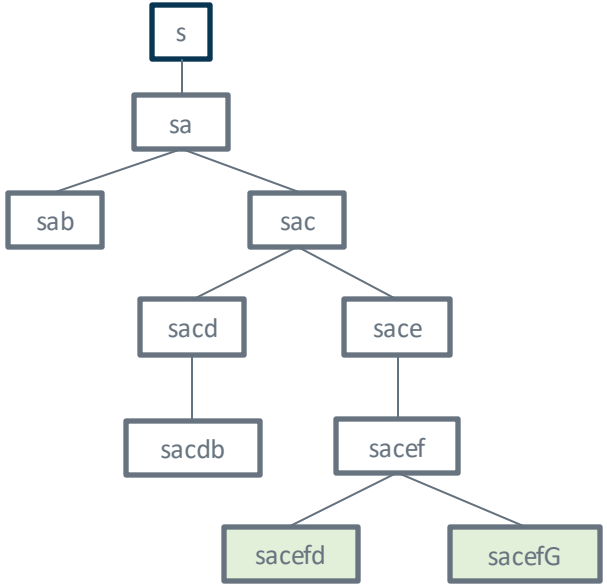




handling cycles

(Memory Efficient) DFS





DFS vs. BFS

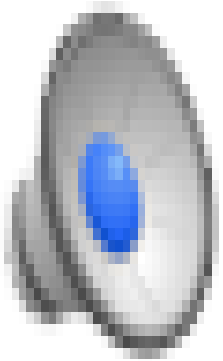
	Time	Space
DFS (memory-efficient version)	$m \cdot b^m$	bm^2
BFS	b^d	b^d
IDS	$O\left(\sum_{i=1}^{d} i \cdot b^i\right) \leq O\left(d \cdot \sum_{i=1}^{d} b^i\right) \leq O(d \cdot b^d)$	bd^2

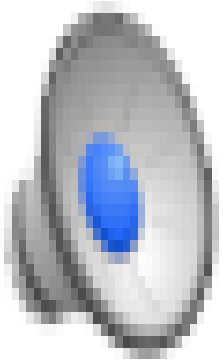
Iterative Deepening Search (IDS)

- Idea: get DFS's space advantage with BFS's time advantage,
  - Run a DFS with depth limit 1. If no solution… b
  - Run a DFS with depth limit 2. If no solution… b^2
  - Run a DFS with depth limit 3. ….. $\vdots$

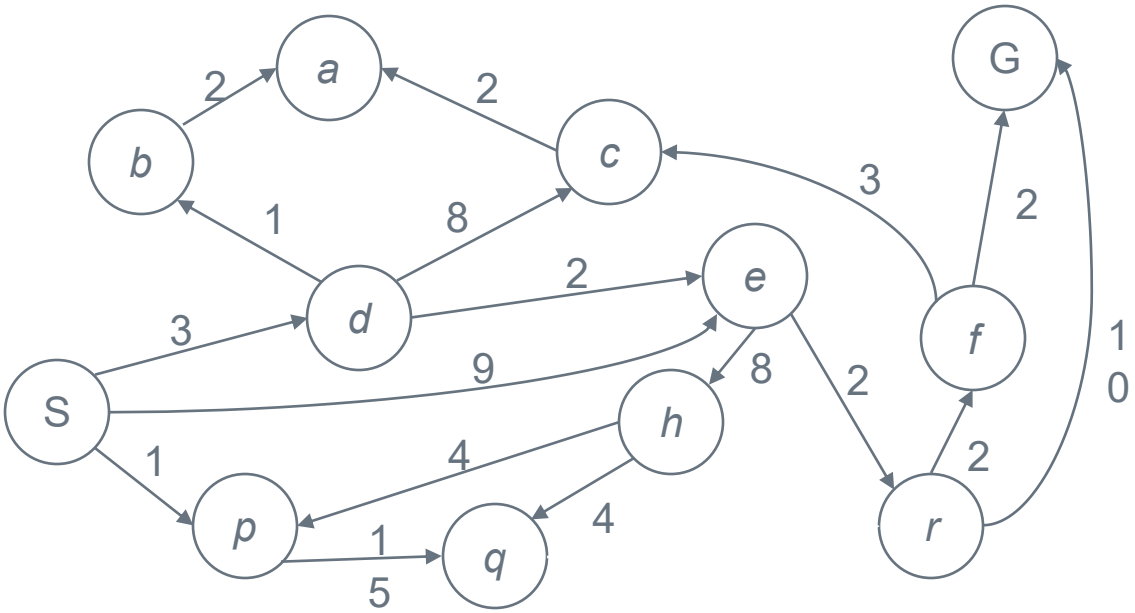
- Isn't that wastefully redundant?
  - Generally most work happens in the last level b^d
  - Branching factor 10, solution 5 deep: limit d.
    - BFS: 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110
    - IDS: 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450

Which One is DFS/BFS?





Cost-Sensitive Search Problem



Uniform Cost Search (Dijkstra)

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

Choose the one with smallest $g(s)$

If s is a goal state, then terminate

For all action a:

$s' \leftarrow \text{succ}(s, a)$

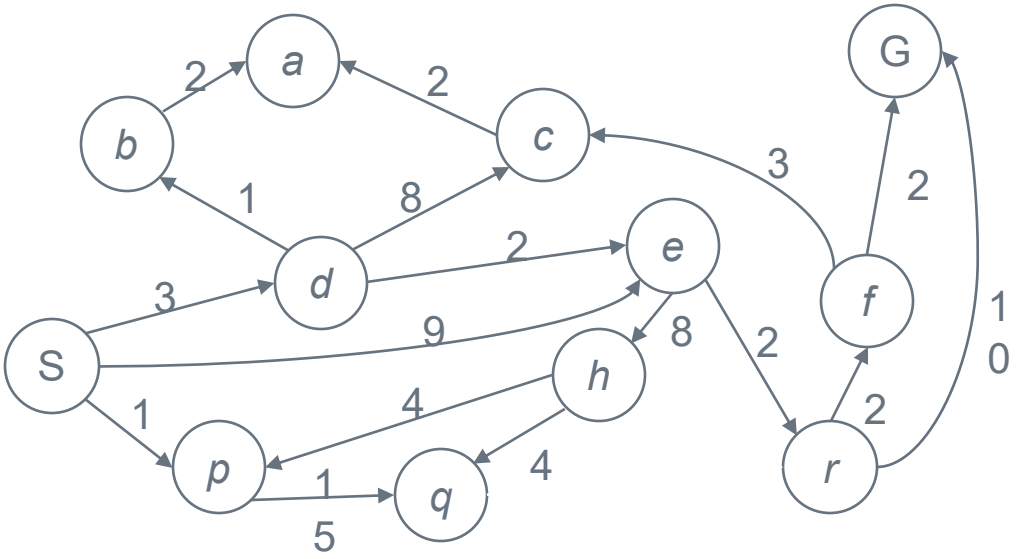
If not **Reached**$[s']$:

Put s' in **Frontier**

Reached$[s']$ ← True

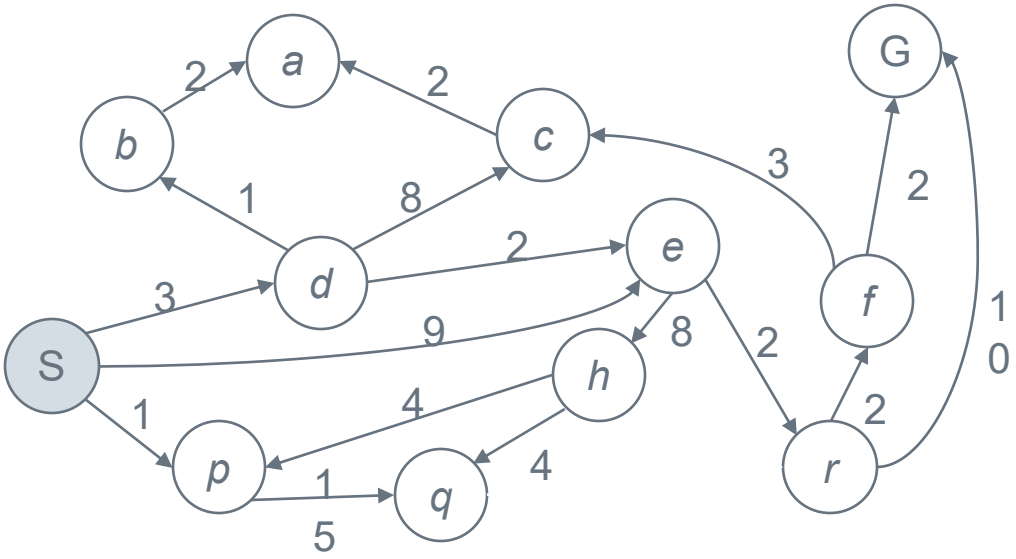
$g(s') \leftarrow \min\{\, g(s'), g(s) + \text{cost}(s, a) \,\}$

UCS



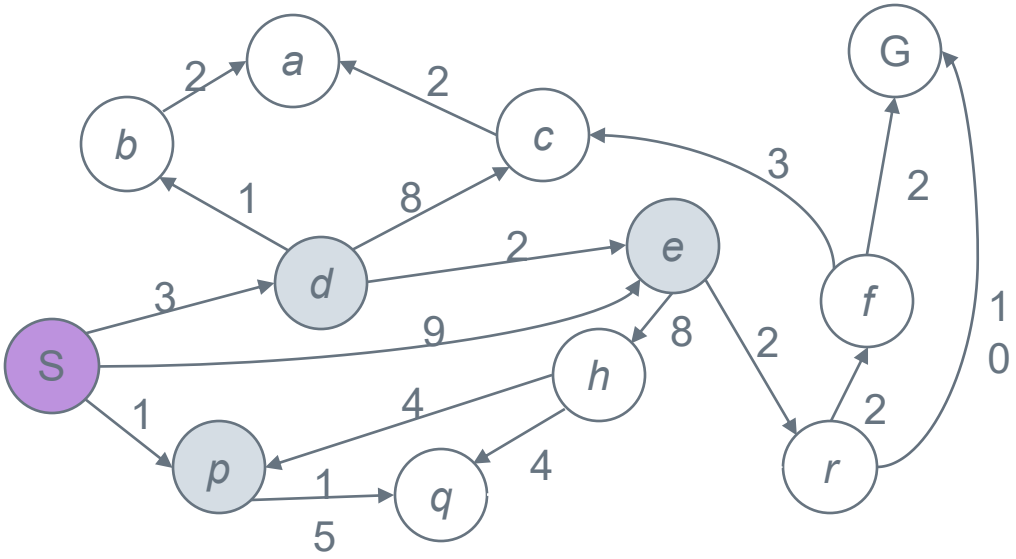
x	$g(x)$
S	
a	
b	
c	
d	
e	
f	
h	
p	
q	
r	
G	

UCS



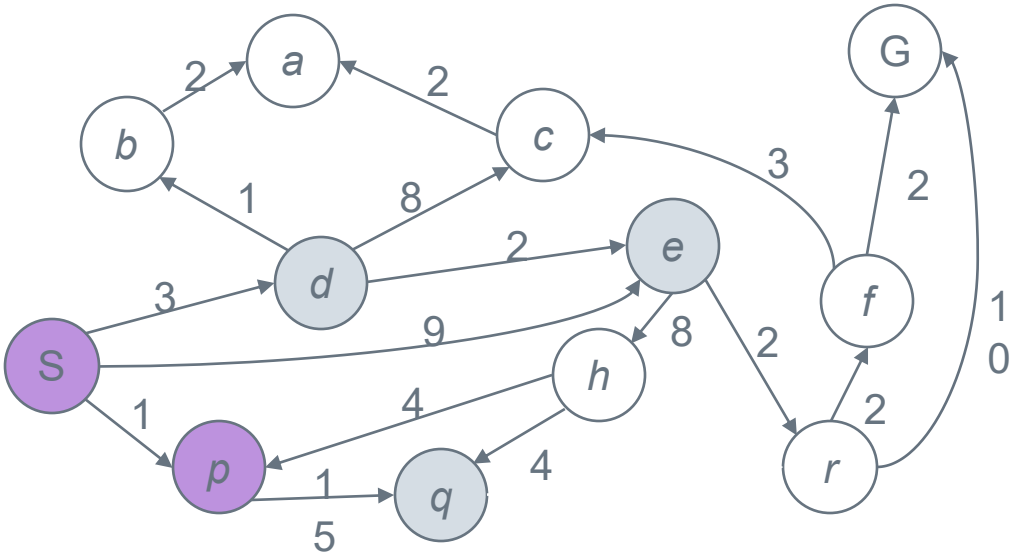
x	$g(x)$
S	0
a	∞
b	∞
c	∞
d	∞
e	∞
f	∞
h	∞
p	∞
q	∞
r	∞
G	∞

UCS



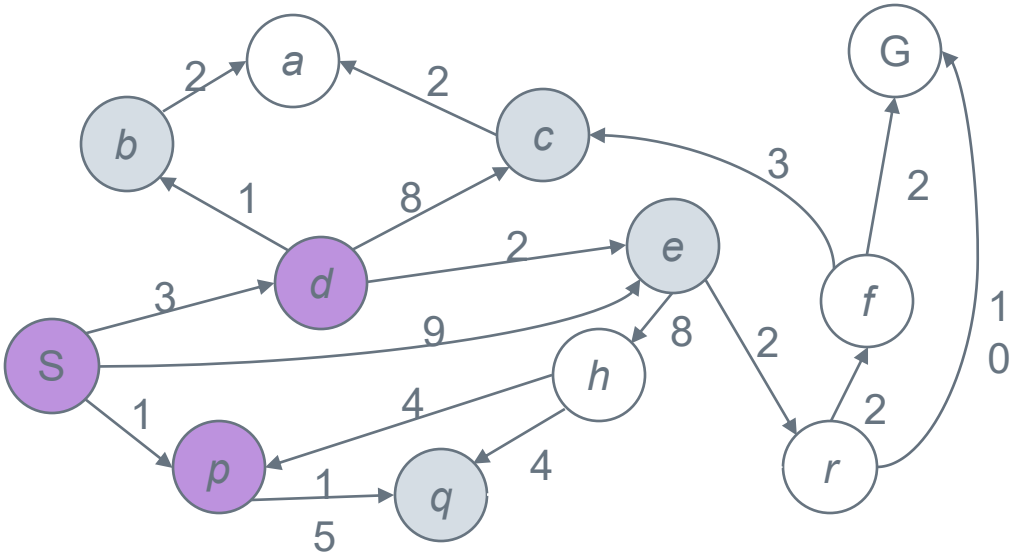
x	$g(x)$
S	0
a	∞
b	∞
c	∞
d	3
e	9
f	∞
h	∞
p	1
q	∞
r	∞
G	∞

UCS



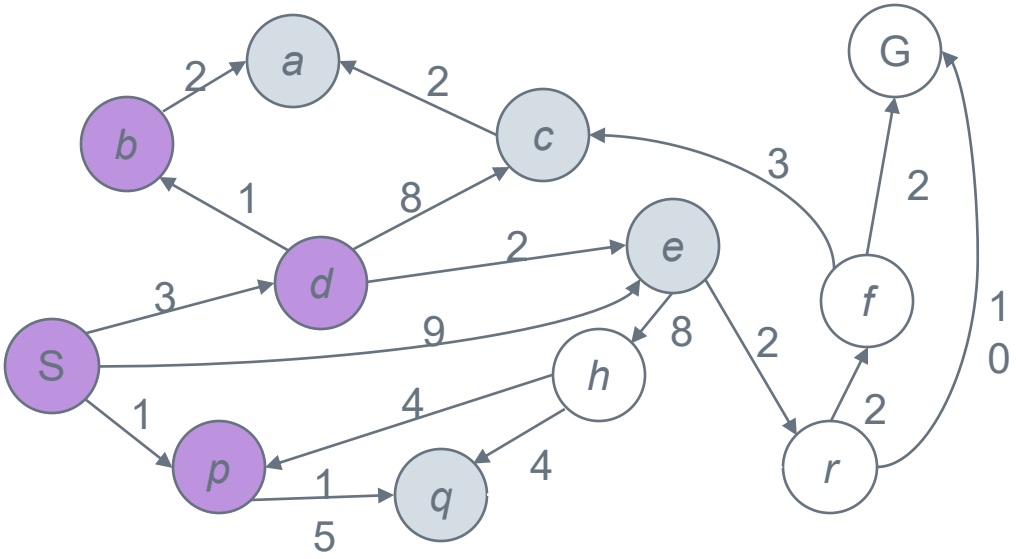
x	$g(x)$
S	0
a	∞
b	∞
c	∞
d	3
e	9
f	∞
h	∞
p	1
q	16
r	∞
G	∞

UCS



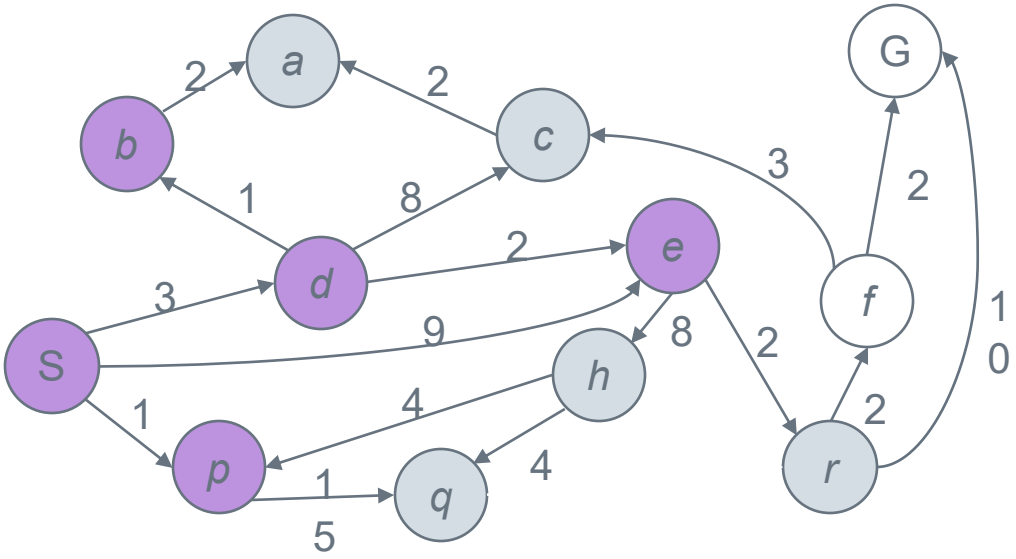
x	$g(x)$
S	0
a	∞
b	4
c	11
d	3
e	5
f	∞
h	∞
p	1
q	16
r	∞
G	∞

UCS



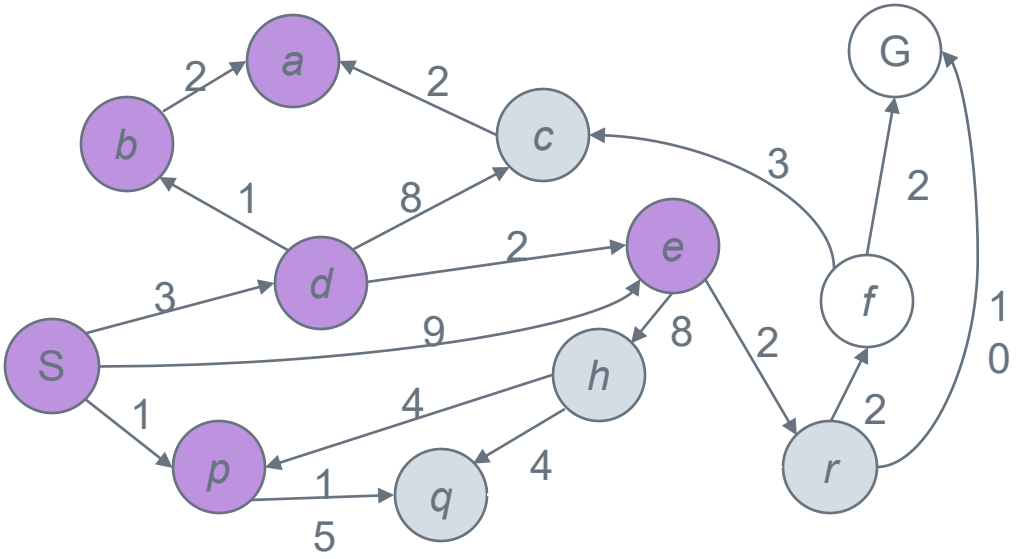
x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	∞
h	∞
p	1
q	16
r	∞
G	∞

UCS



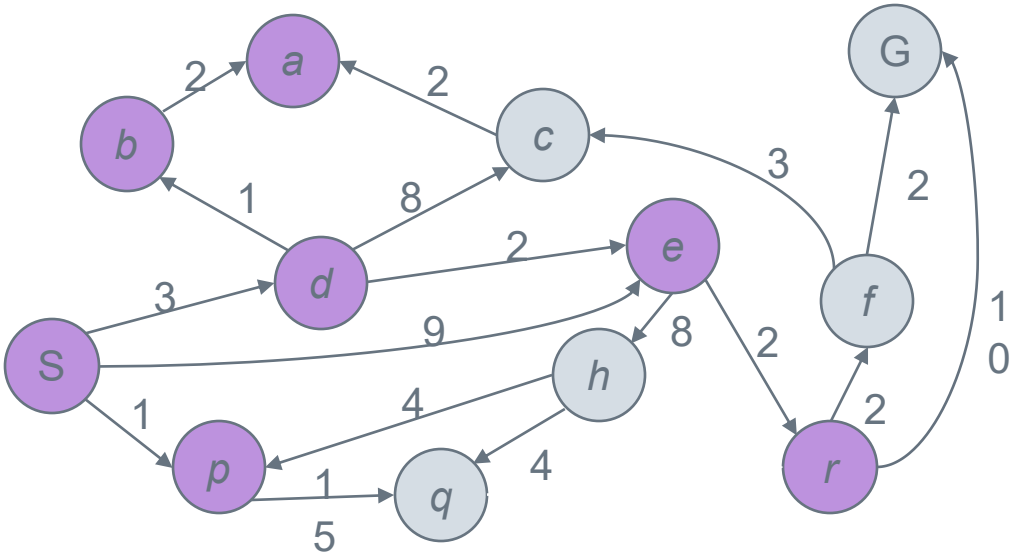
x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	∞
h	13
p	1
q	16
r	7
G	∞

UCS



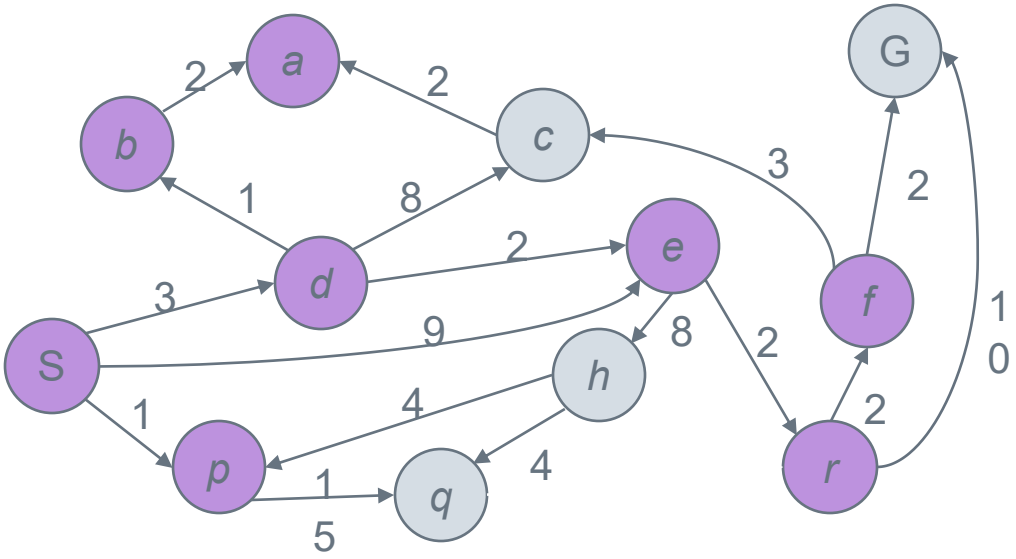
x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	∞
h	13
p	1
q	16
r	7
G	∞

UCS



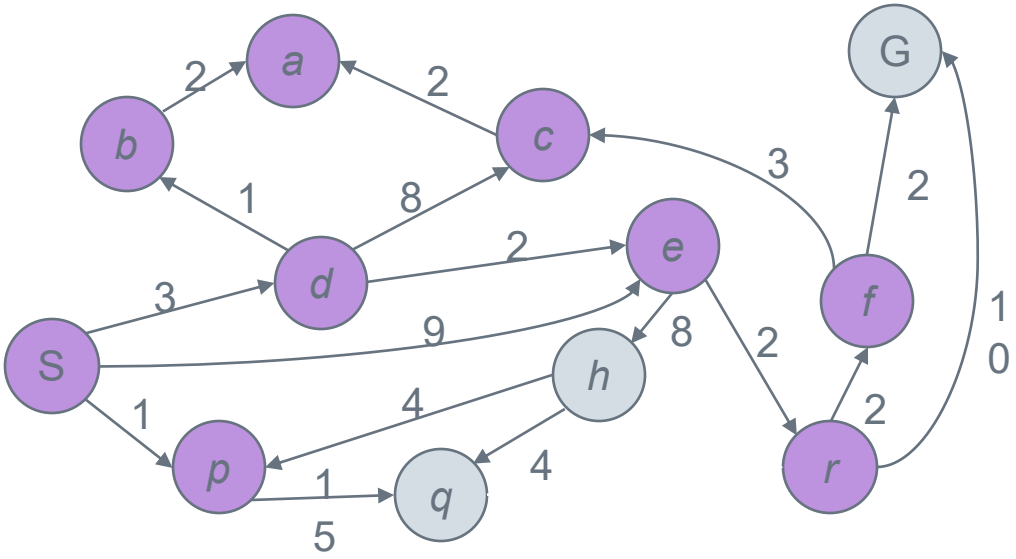
x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	17

UCS



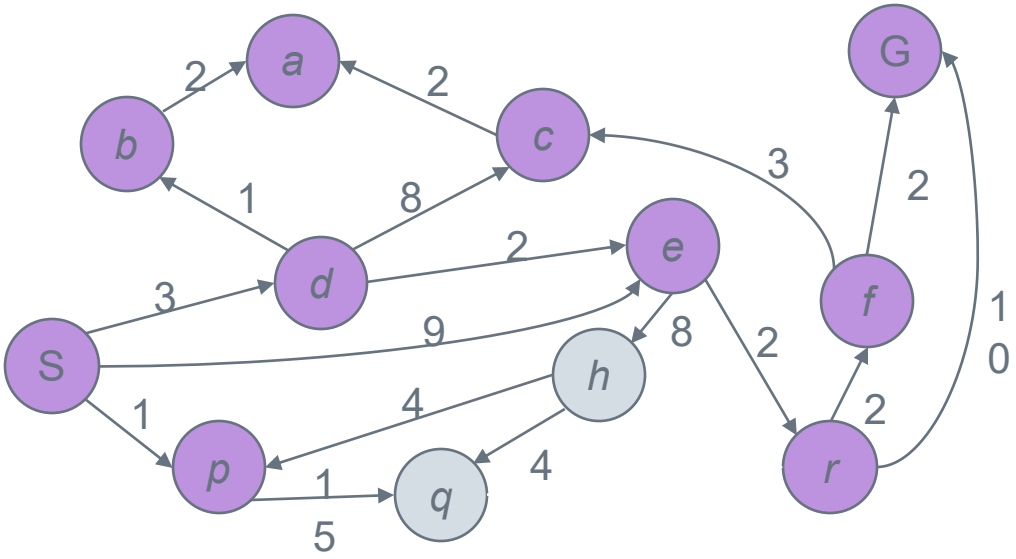
x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	11

UCS



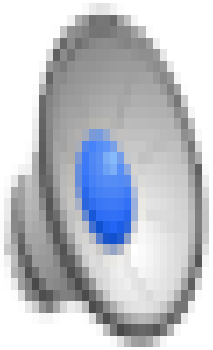
x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	11

UCS

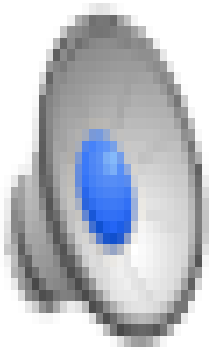


x	$g(x)$
S	0
a	6
b	4
c	11
d	3
e	5
f	9
h	13
p	1
q	16
r	7
G	11

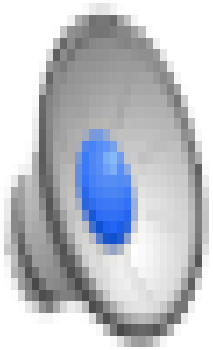
DFS/BFS/UCS? (Deep/light blue → high/low cost)



DFS/BFS/UCS? (Deep/light blue → high/low cost)

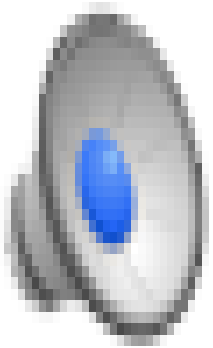


DFS/BFS/UCS? (Deep/light blue → high/low cost)

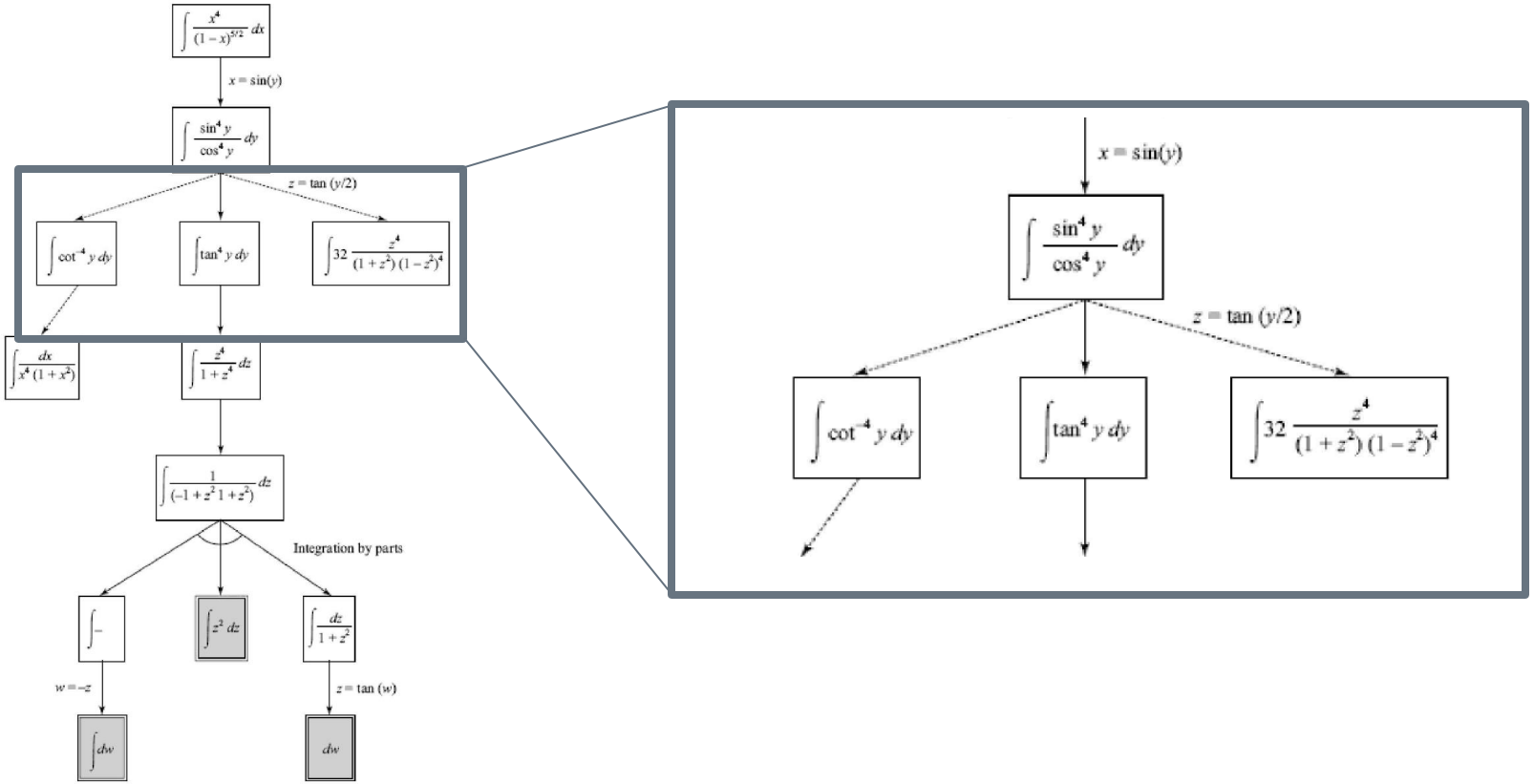


Informed Search

Inefficiency of the Search Algorithms We See So Far



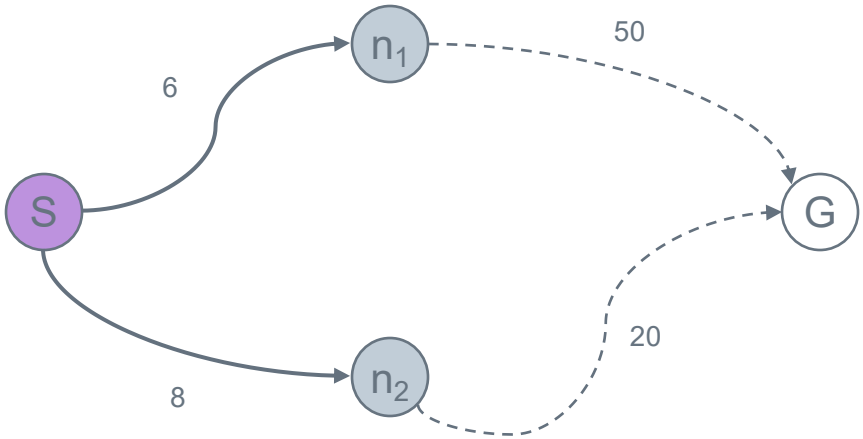
Inefficiency of the Search Algorithms We See So Far



Heuristic Function

Suppose we have some **“guess”** for the distance from every node to the goal.

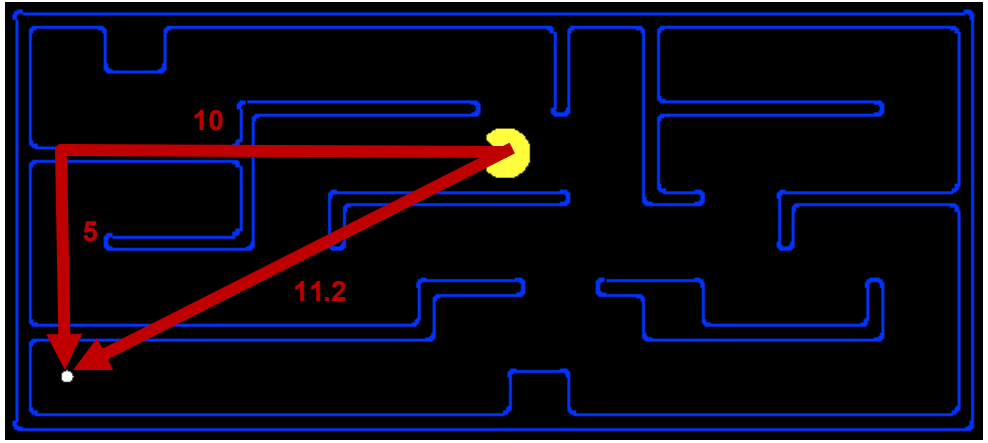
Can we leverage it to accelerate the search?



Heuristic Function

Having a heuristic function that accurately predict the distance might be impossible.

However, some function that **correlates** with the true distance may be easy to find.



Suppose we have a heuristic function $h(s)$.

Greedy Best-First Search

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

Choose the one with smallest $h(s)$

For all action a:

$s' \leftarrow \text{succ}(s, a)$

If not Reached[s']:

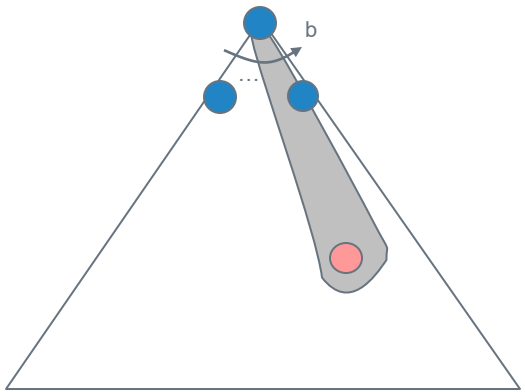
If s' is a goal state, then terminate

Push s' to **Frontier**

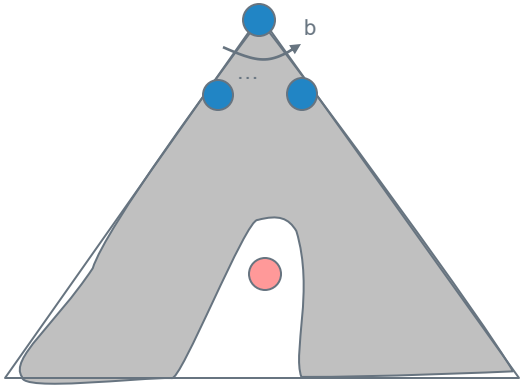
Reached[s'] ← True

Greedy Best-First Search

- If the heuristic is good
  - Take us directly to the goal
  - Like a nicely-guided DFS



- In the worst case
  - Take us to the wrong way
  - Like a badly-guided DFS



Suppose we have a heuristic function $h(s)$

A* Search: Combining Greedy and UCS

Backward cost **Forward cost**

Frontier ← { initial_state }

While **Frontier** is not empty:

Pop a node s from **Frontier**

Choose the one with smallest $g(s) + h(s)$

If s is a goal state, then terminate

For all action a:

$s' \leftarrow \text{succ}(s, a)$

If not Reached[s']:

Push s' to **Frontier**

Reached[s'] ← True

$g(s') \leftarrow \min\{\, g(s'), g(s) + \text{cost}(s, a) \,\}$

UCS is a special case with $h(s) = 0$

A* Search

Evaluation functions:

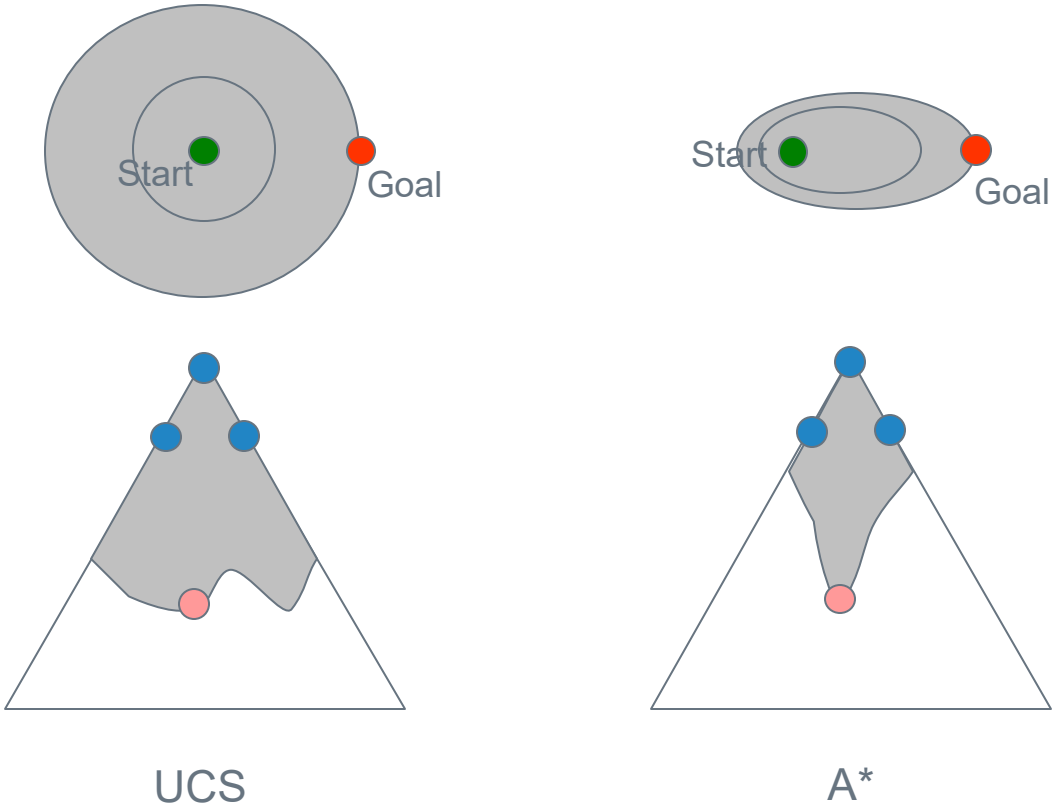
Greedy (Greedy Best-First) Search: $h(s)$

Uniform Cost Search: $g(s)$

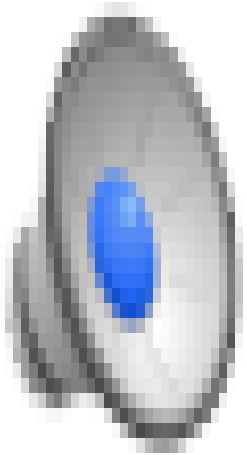
A* Search: $g(s) + h(s)$

Weighted A* Search: $g(s) + w \cdot h(s)$ for some $w \in (0, \infty)$

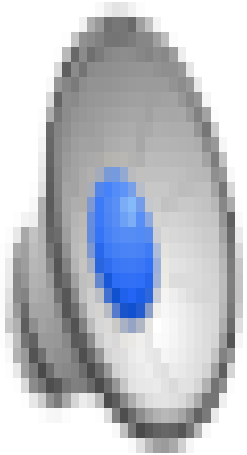
UCS vs. A*



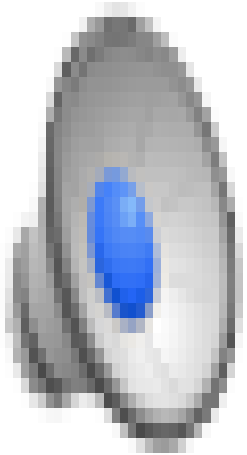
UCS / Greedy Best-First / A* ?



UCS / Greedy Best-First / A* ?



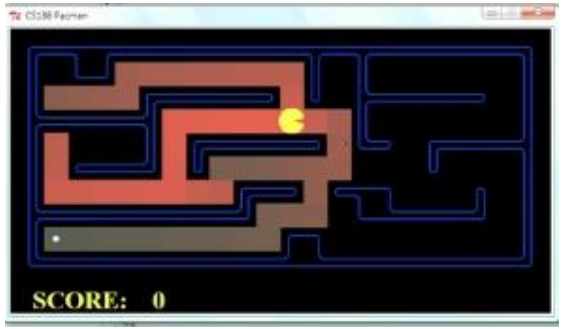
UCS / Greedy Best-First / A* ?



Comparison

Greedy

UCS



A*

The Optimality of A*

Theorem

If the heuristic function is **consistent**, then A* returns minimum-cost solution.

Proof A* is equivalent to UCS with $\widetilde{\text{cost}}(s \to s') := \underbrace{\text{cost}(s \to s') + h(s') - h(s)}_{\text{consistency}}$ ≥ 0

Total modified cost of any path $s_1 \to s_2 \to \cdots \to s_m \to G$ is

$$\widetilde{\text{cost}}(s_1 \to s_2) + \widetilde{\text{cost}}(s_2 \to s_3) + \cdots + \widetilde{\text{cost}}(s_m \to G)$$

$$= [\text{cost}(s_1 \to s_2) + h(s_2) - h(s_1)] + [\text{cost}(s_2 \to s_3) + h(s_3) - h(s_2)] + \cdots + [\text{cost}(s_m \to G) + h(G) - h(s_m)]$$

$$= \text{cost}(s_1 \to s_2) + \text{cost}(s_2 \to s_3) + \cdots + \text{cost}(s_m \to G) - h(s_1) + h(G)$$

Since $\widetilde{\text{cost}}(s \to s') \geq 0$ by the consistency of h, A*'s optimality follows UCS's optimality under non-negative cost.

Definitions

A heuristic function h is called **admissible** if for all s

$$0 \leq h(s) \leq h^{\star}(s)$$

where $h^{\star}(s)$ is the true minimum distance from s to goal.

Theorems

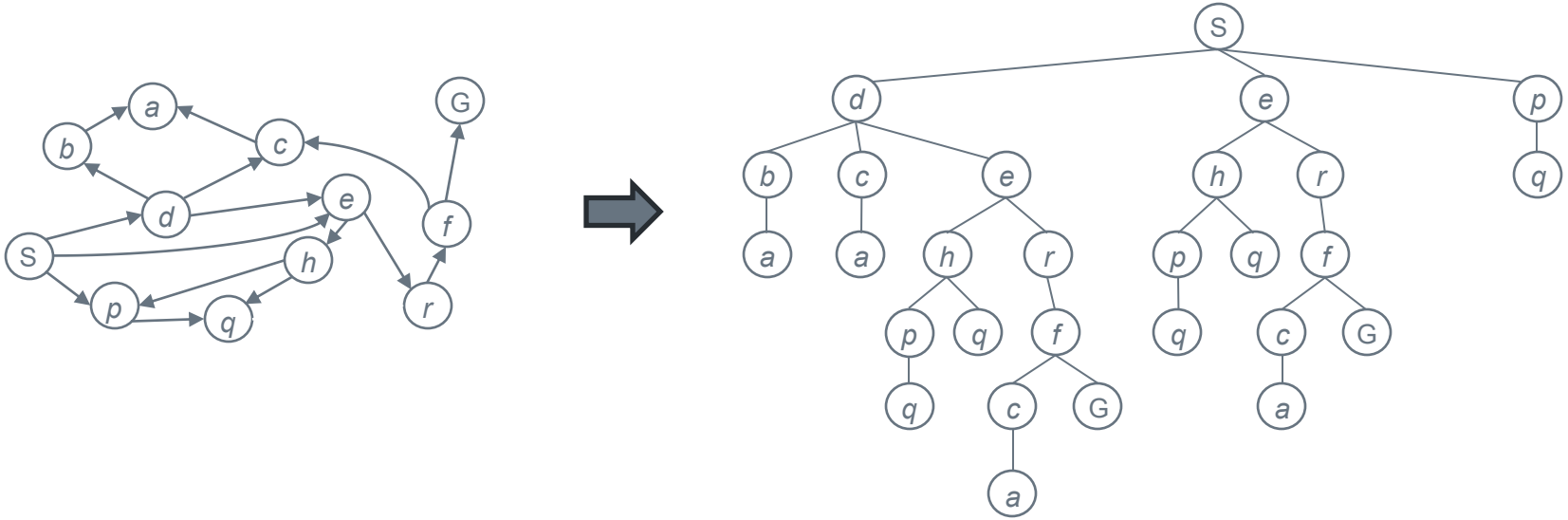
If h is consistent, then it is also admissible.

If the heuristic function is **admissible** and **the graph is a tree,** then A* returns minimum-cost solution.

The Optimality of A*

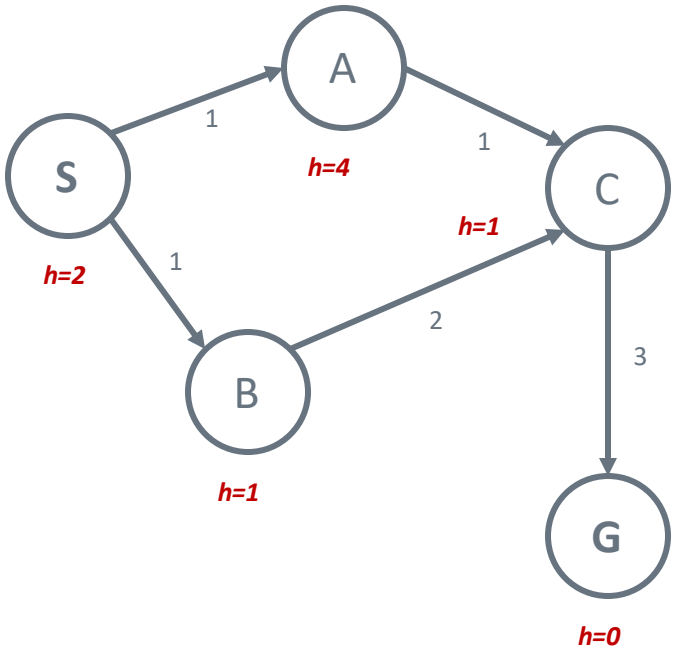
For general graphs, we can **treat it like a tree** by omitting the condition **“if not Reached[s’]”** in the graph search algorithm (like what we did for memory-efficient DFS).

This allows us to apply the theorem in the previous page.

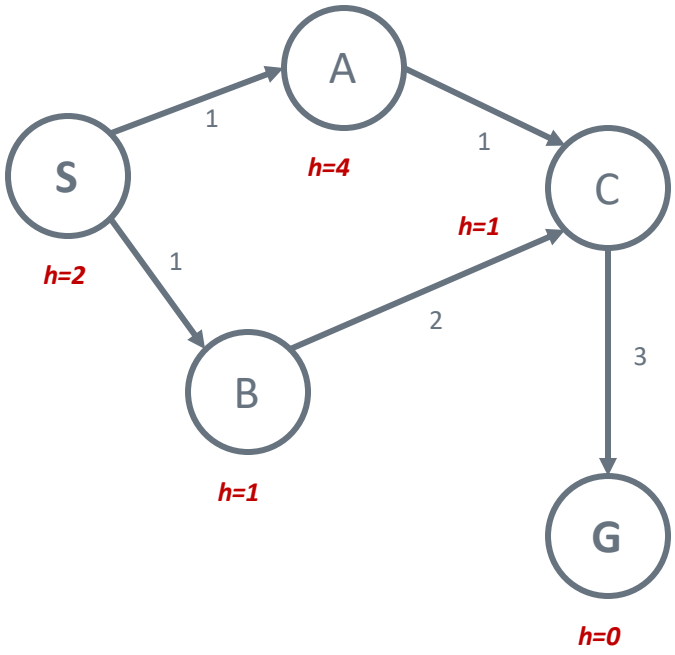


Admissible but not Consistent

$\cancel{h(A) \geq c(A \to C) + h(C)}$



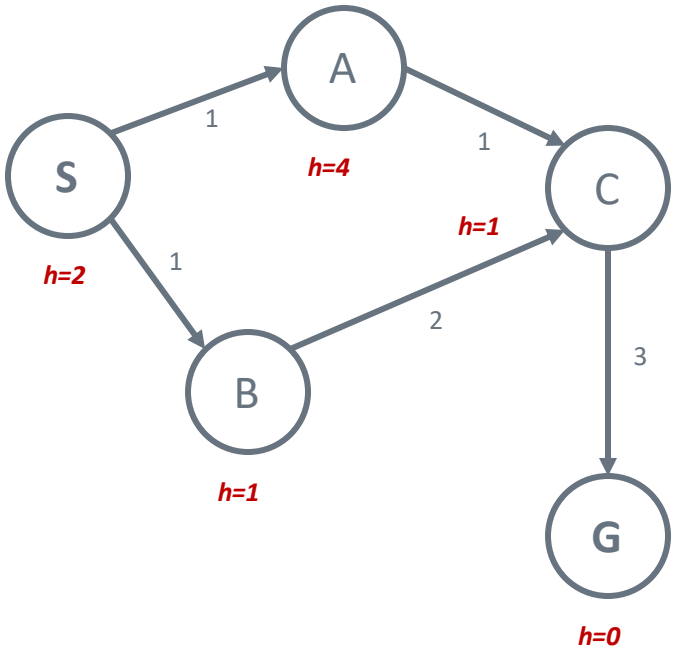
Admissible but not Consistent

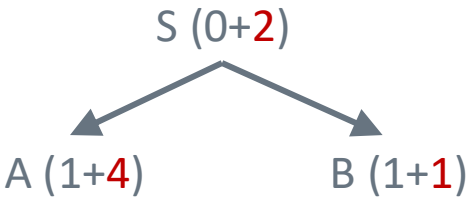


S (0+2)

Expanded = { }

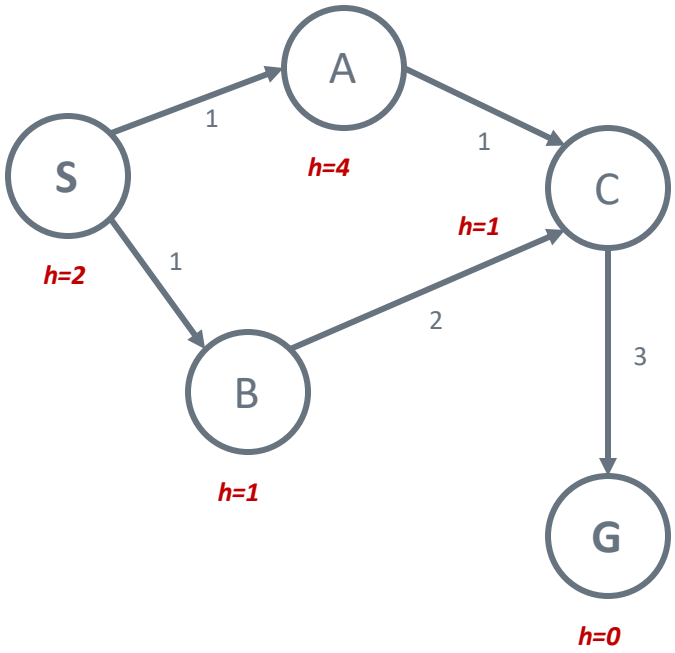
Admissible but not Consistent

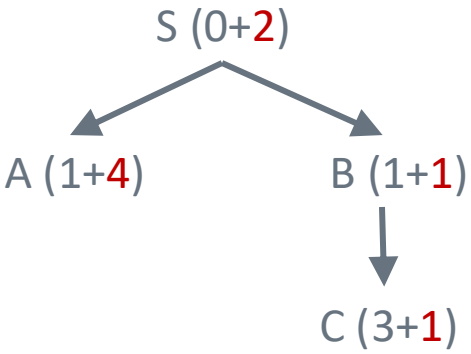




Expanded = { S }

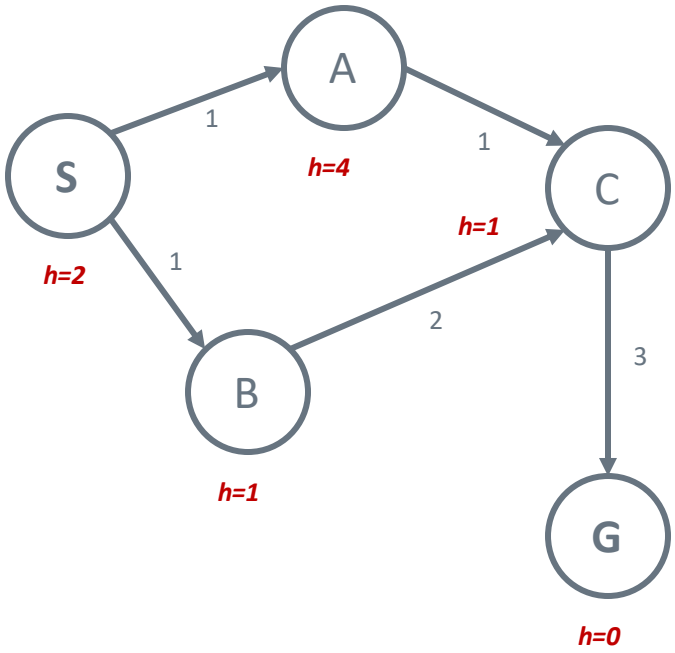
Admissible but not Consistent

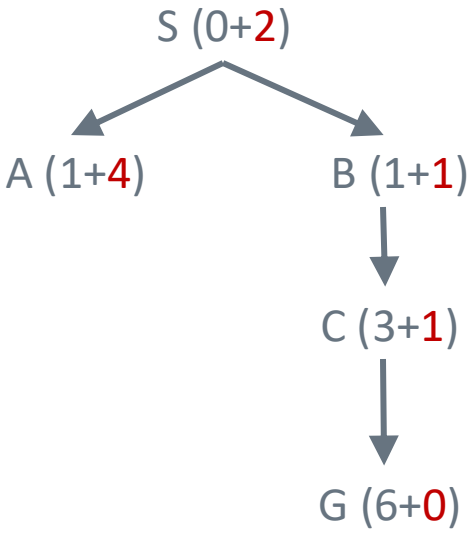




Expanded = { S, B }

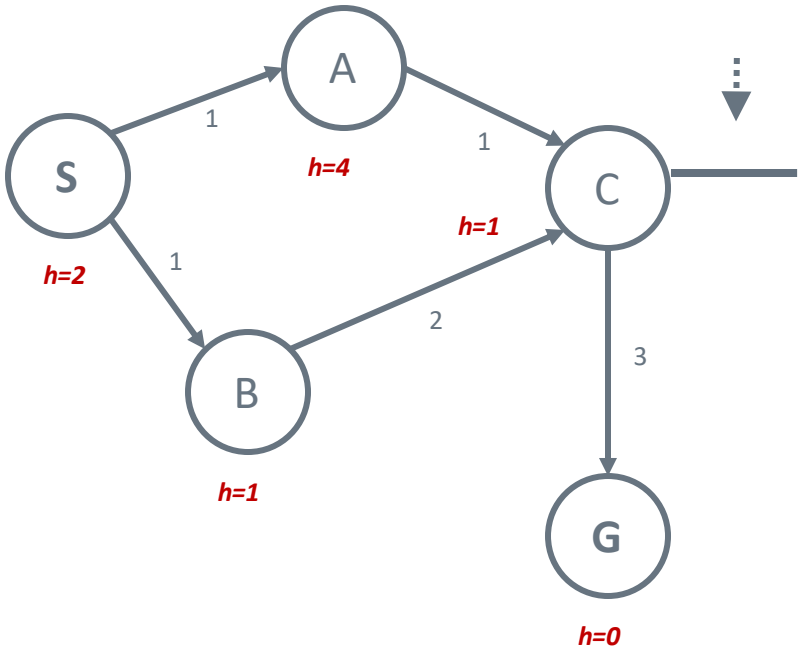
Admissible but not Consistent

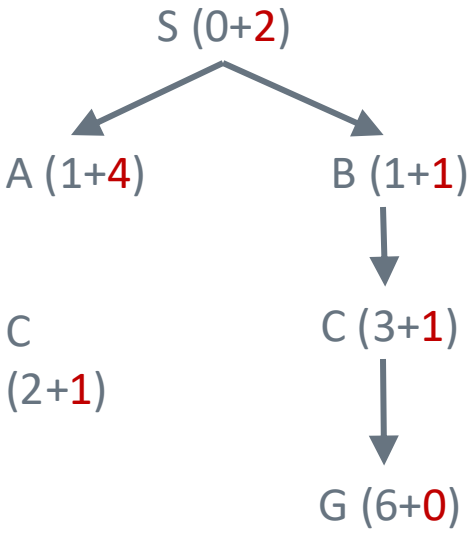




Expanded = { S, B, C }

Admissible but not Consistent





Expanded = { S, B, C }

Discussion...

Q n A